

Capitolo 1

Internet of Things (IoT)

The **Internet of Things (IoT)** refers to the integration of information about physical objects into the Internet. In other words, even trivial everyday items—such as thermometers, parking spots, or watches—can now connect and interact online.

The key idea is to make computing **ubiquitous and invisible to users**: IoT devices work without requiring explicit user actions. Instead, they rely on sensing, rules, and automated responses to environmental conditions.

A well-known definition by Kevin Ashton, who coined the term *Internet of Things*, captures the essence:

In the twentieth century, computers were brains without senses – they only knew what we told them.

In the twenty-first century, because of the Internet of Things, computers can sense things for themselves.

1.1 Definitions of IoT

Several organizations provide complementary definitions:

- **General definition:** “A global infrastructure for the information society, enabling advanced services by interconnecting physical and virtual things based on interoperable communication technologies”.
- **IEEE:** “A network of items — each embedded with sensors — which are connected to the Internet”.
- **ITU:** “Available anywhere, anytime, by anything and anyone”.
- **IETF:** IoT connects electronic, electrical, and even non-electrical objects to provide contextual services, enabled by technologies such as RFID, sensors, and actuators.
- **NIST:** Defines IoT as a form of **Cyber-Physical Systems (CPS)**, connecting smart devices in sectors like transport, energy, manufacturing, and healthcare.

- **W3C**: Stresses the role of web technologies in the *Web of Things*, where physical objects and their virtual counterparts can interact via HTTP, REST, and JavaScript APIs.

A more **formal and precise definition** emphasizes that:

- *No IP means no IoT*. Devices must be addressable through IP, ideally IPv6.
 - IoT is the paradigm in which every “thing” (real or virtual) is assigned an IP address and can be reached via the standard Internet Protocol stack.
-

1.2 Functional Overview of IoT

IoT systems can be described in terms of four main functions:

1. Sensing (Data Collection)

- Sensors act as the interface between the physical and digital world, producing data.
- Collection modes include:
 - **Periodic** (regular sampling),
 - **On-demand** (upon request),
 - **Event-driven** (alerts when conditions are met).
- Data must be enriched with **metadata** (type, location, context, structural relationships).

2. Processing and Visualization

- Processing can be:
 - **Control loops** (simple, real-time adjustments to reach a setpoint),
 - **Analytics & machine learning** (based on historical data and patterns).
- Typical processing steps: sampling, aliasing, quantization, hysteresis, calibration, error propagation, optimization, and prediction.
- Results are often presented in **dashboards** or as a **digital twin** of the system.

3. Acting

- The ultimate goal of IoT is action: applying insights to the physical world.
- Actions may be:
 - Direct (automatic actuation, e.g., switch on AC if temperature > threshold),
 - Indirect (recommendations to operators, process optimization).
- IoT also helps identify anomalies and trigger corrective actions.

4. System Overview

IoT architecture is typically layered:

- **Sensors** → collect data
 - **Gateways (Edge/Fog nodes)** → aggregate, filter, and connect to the wider Internet (via Wi-Fi, Bluetooth, LoRa, NB-IoT, etc.)
 - **Cloud** → large-scale processing, ML/AI, visualization, and storage
 - **Applications** → services improving quality of life in multiple domains.
-

1.3 Applications of IoT

IoT improves life quality across many sectors:

- **Industry 4.0 (Industrial IoT):**
 - Safety improvements, predictive maintenance, energy management, supply chain optimization, and data-driven decision-making.
 - Examples:
 - ✱ *Tenaris*: smart sensors for real-time motor monitoring
 - ✱ *Unilever*: digital twins of production facilities
 - ✱ *Volkswagen*: Industrial Cloud integrating 120+ factory sites
- **Smart Retail:**
 - Self-checkout, store layout optimization, marketing personalization, inventory management, supply chain monitoring.
 - Examples: *AmazonGo* (cashierless shopping), *Kroger* (smart shelves), *Rebecca Minkoff* (smart mirrors).
- **Healthcare:**
 - Remote patient monitoring, medical alert systems, diagnostics, continuous glucose monitoring, cardiac sensors.
- **Smart Cities & Homes:**
 - Smart lighting, HVAC, security cameras, air quality monitors, parking, appliances like *Philips Hue* or *Gardena irrigation*.
- **Transportation & Automotive:**
 - Autonomous driving, predictive vehicle maintenance, fleet management, usage-based insurance. Examples: *Waymo*, *Tesla Autopilot*, *BMW remote parking*.

Capitolo 2

IoT Sensors

- Sensors can measure some quality in the environment
- Actuators can perform actions in the environment
- Some sensors are capable of **local operations** (the sensor elaborates the data, real time commands for example)
- Some devices connect directly to the internet and communicate with cloud applications
 - Energy constraints may require external computation platform (because they are battery-powered)
 - *Examples: security cameras, fire sensors, thermostats, appliances, power meters.*
- Trade-offs include **security, latency, computational capacity, and energy consumption.**

Capitolo 3

Gateways

- Sensors usually connect to the Internet via **intermediaries** such as gateways.
- They are more powerful devices
- *Typical connections: ZigBee, IEEE 802.15.4 variants, Bluetooth, low-power Wi-Fi, LoRa, NB-IoT.*
- Gateways provide:
 - Wide-area connectivity
 - Edge processing like *protocol conversion, data storage/filtering, event processing, analytics*

()
*

Com-
mu-
ni-
ca-
tion
0.0556Layers

()
*

Com-
mu-
ni-
ca-
tion
0.0556Layers

()
*
0.0556#
The
Cloud
-

The
“back-
end”
for
IoT
pro-
ces-
sing.
-

Not
man-
da-
to-
ry
-

Ag-
gre-
ga-
tes
da-
ta
from
di-
ver-
se
sour-

Capitolo 4

Control Plane

- **Data plane / user plane:** collect, process, and act on data.
- **Control plane:** ensures infrastructure functionality and security.
 - Service configuration & aggregation
 - Problem management
 - Quality management
 - Resource provisioning
 - Trouble & anomaly management (security)
 - Performance management

()

*

0.0556##

Why

IoT

To-

day?

()

*

0.0556-

A

con-

ver-

gen-

ce

of

tech-

no-

lo-

gi-

cal

and

in-

fra-

struc-

tu-

re

fac-

tors:

-

In-

du-

stry

4.0

and

di-

gi-

ta-

li-

za-

tion

-

Af-

for-

da-

ble

and

di-

ver-

se

sen-

sors

-

12

Mi-

nia-

tu-

Capitolo 5

Main Problems in IoT

5.1 Energy Problem

- Sensors often in **hard-to-reach locations** → battery-powered for months/years.
 - Possible use of solar panels or energy harvesting (size/cost limitations).
 - Energy affects communication, computation, sensing, actuators.
 - Optimizations include:
 - Fewer transmissions (duty cycle, decided by law)
 - Reducing transmitted bytes
 - Minimizing on-board processing
 - Energy-saving mechanisms (sleep, idle, discontinuous reception).
-

5.2 Security Problem

- IoT devices are simple → hard to secure.
 - Challenges:
 - Strong cryptography infeasible (energy constraints).
 - Difficult to patch/update vulnerabilities.
 - Connected by design to the open Internet.
 - Often managed by non-IT users.
 - Privacy concerns:
 - Location data = movement/activity tracking
 - Home automation reveals daily habits
 - Breaches can harm **reputation and financial stability**.
-

5.3 Scalability Problem

- Traditional cellular/wireless networks designed for **few devices with high data needs**.
 - IoT requires support for **millions of devices with small data flows** (congestion).
 - New IoT-specific technologies and protocols are needed.
-

5.4 Reliability Problem

- Key in industrial and safety-critical systems.
- Issues:
 - Identifying anomalous/hazardous situations
 - Handling anomalies across single or multiple sensors
 - Trusting anomaly detection when sensors are unreliable

Capitolo 6

IoT System Architecture

6.1 Communication Paradigms

- **Client/Server**: sources explicitly know the destination of their messages.
- **Publish/Subscribe**: sources publish to a logical channel; interested destinations subscribe (one-to-many, no explicit sessions).
- Typical IoT traffic flows:
 - **Asynchronous events** (uplink)
 - **Periodic or on-demand measurements** (uplink)
 - **Commands** from processing platforms (downlink)

()

*

Pro-
per-
ties
of
an
IoT
Sy-
stem
-

**Tru-
st-
wor-
thi-
ness:**
avai-
la-
bi-
li-
ty,
re-
si-
lien-
ce,
con-
fi-
den-
tia-
li-
ty,
in-
te-
gri-
ty,
sa-
fe-
ty.
-

**Ar-
chi-
tec-
tu-
ral:**
com-
po-
sa-
bi-
li-
ty,
mo

()
*

Pro-
per-
ties
of
an
IoT
Sy-
stem
-

**Tru-
st-
wor-
thi-
ness:**
avai-
la-
bi-
li-
ty,
re-
si-
lien-
ce,
con-
fi-
den-
tia-
li-
ty,
in-
te-
gri-
ty,
sa-
fe-
ty.
-

**Ar-
chi-
tec-
tu-
ral:**
com-
po-
sa-
bi-
li-
ty,
mo

()
*

Pro-
per-
ties
of
an
IoT
Sy-
stem
-

**Tru-
st-
wor-
thi-
ness:**
avai-
la-
bi-
li-
ty,
re-
si-
lien-
ce,
con-
fi-
den-
tia-
li-
ty,
in-
te-
gri-
ty,
sa-
fe-
ty.
-

**Ar-
chi-
tec-
tu-
ral:**
com-
po-
sa-
bi-
li-
ty,
mo-

6.2 Network Architectures

- **One-level:** endpoints use full IP stack (e.g., cellular).
- **Two-level (1b):** endpoints resource-constrained, gateway manages protocols (LP-WA, Low Power Wide Area: LoRa, SigFox, NB-IoT). *Target of the course*
- **Two-level (1c):** endpoints support TCP/IP, traffic managed by router (Ethernet/Wi-Fi)

()
*

WANs

-

TCP/IP
net-
works,
shared
among
users.

-

Pu-
blic
by
na-
tu-
re.

-

VPNs:
re-
ser-
ved
band-
width/security
for
de-
fi-
ned
groups,
with
pri-
va-
te
IP
ad-
dres-
sing

-

Users
ou-
tsi-
de
of
a
group

can-
not
com-
mu

()
*

WANs

-

TCP/IP
net-
works,
shared
among
users.

-

Pu-
blic
by
na-
tu-
re.

-

VPNs:
re-
ser-
ved
band-
width/security
for
de-
fi-
ned
groups,
with
pri-
va-
te
IP
ad-
dres-
sing

-

Users
ou-
tsi-
de
of
a
group

can-
not
com-
mu

Capitolo 7

Endpoints

7.1 Sensors

- Devices detecting stimuli → electrical/processable output.
- Types: pressure, temperature, biometric, chemical, acoustic, optical, positioning, etc.
- Three classifications:
 - **Fixed**: stable, not moving like domotics
 - **Mobile**: always moving (GPS, wearables)
 - **Nomadic**: they move but then they stop somewhere else (portable medical devices, field sensors)
- **Active vs Passive** sensors
 - **Active**: need to run a current or send a signal to get a measurement (e.g., clocks, strain gauges, radar).
 - **Passive**: use the physical phenomenon itself (e.g., piezoaccelerometers, GPS, thermocouples).

7.1.1 Actuators

- Output elements performing actions.
- **Digital**: on/off switching.
- **Analog**: continuous signals (e.g., motor speed control).

7.1.2 Gateways

- Bridge sensors → higher-level processing/cloud.
- Provide connectivity, data collection, filtering, storage, event processing, optional analytics

7.1.3 Fog Nodes

- “Bring the cloud to the ground.”
- More powerful than gateways, allow **local analytics with lower latency**.

```
()  
*  
#  
Da-  
ta  
pla-  
ne  
func-  
tions  
##  
Da-  
ta  
ac-  
qui-  
si-  
tion  
and  
pro-  
ces-  
sing  
0.0556sing  
()  
*  
0.0556###  
Sam-  
pling
```

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 sing
 0.0556

()
 *
 0.0556-

Fir-
 st,
 we
 sam-
 ple:
 we
 ta-
 ke
 (re-
 gu-
 lar)
 mea-
 su-
 re-
 men-
 ts
 over
 ti-
 me,
 and
 re-
 pre-
 sent
 the
 si-
 gnal
 with
 tho-
 se

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 0.0556sing

()
 *
 0.0556###
 Alia-
 sing
 -

If
 we
 do
 not
 sam-
 ple
 “enou-
 gh,”
 we
 ha-
 ve
 alia-
 sing:
 pat-
 terns
 in
 the
 sam-
 pled
 ver-
 sion.
 -

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 0.0556sing

()
 *
 0.0556###
 Quan-
 ti-
 za-
 tion
 -

Di-
 gi-
 tal
 va-
 lues
 are
 fi-
 ni-
 te:
 we
 can-
 not
 re-
 pre-
 sent
 con-
 ti-
 nuous
 va-
 lues
 pre-
 ci-
 se-
 ly!

```
()  
*  
#  
Da-  
ta  
pla-  
ne  
func-  
tions  
##  
Da-  
ta  
ac-  
qui-  
si-  
tion  
and  
pro-  
ces-  
sing  
0.0556  
()  
*  
0.0556
```

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 sing
 0.0556sing

()
 *
 0.0556###
 Sa-
 tu-
 ra-
 tion
 -

Sen-
 sors
 of-
 ten
 ha-
 ve
 an
 up-
 per
 li-
 mit,
 which
 is
 ei-
 ther
 phy-
 si-
 cal
 or
 dic-
 ta-
 ted
 by
 the

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 0.0556sing

()
 *
 0.0556###
 Hy-
 ste-
 re-
 sis
 -

So-
 me-
 ti-
 mes
 sen-
 sors
 ta-
 kes
 a
 whi-
 le
 to
 track
 the
 pro-
 cess
 be-
 cau-
 se
 the
 phy-
 si-
 cal
 na-

```
()
*
#
Da-
ta
pla-
ne
func-
tions
##
Da-
ta
ac-
qui-
si-
tion
and
pro-
ces-
sing
0.0556
```

```
()
*
0.0556-
```

Exam-
ple:
When
you
ap-
ply
pres-
su-
re
to
a
sen-
sor
and
then
re-
lea-
se
that
pres-
su-
re,
the
sen-
sor's
out-
put
should
in

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 sing
 0.0556sing

()
 *
 0.0556###
 Non-
 linearities
 -

Ideal-
 ly,
 sen-
 sors
 should
 al-
 so
 ha-
 ve
 li-
 near
 re-
 spon-
 ses:
 in
 rea-
 li-
 ty,
 the-
 re
 are
 of-
 ten
 non-
 linear
 ef-

```
()  
*  
#  
Da-  
ta  
pla-  
ne  
func-  
tions  
##  
Da-  
ta  
ac-  
qui-  
si-  
tion  
and  
pro-  
ces-  
sing  
0.0556  
()  
*  
0.0556-  
  
Hard  
to  
pre-  
dict
```

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 sing
 0.0556sing

()
 *
 0.0556-

Exam-
 ple:
 A
 li-
 near
 sen-
 sor's
 out-
 put
 will
 chan-
 ge
 by
 equal
 amoun-
 ts
 for
 equal
 pres-
 su-
 re
 chan-
 ges
 across
 its
 en-
 ti-
 re
 ran-

```
()  
*  
#  
Da-  
ta  
pla-  
ne  
func-  
tions  
##  
Da-  
ta  
ac-  
qui-  
si-  
tion  
and  
pro-  
ces-  
sing  
0.0556  
()  
*  
0.0556###  
Ca-  
li-  
bra-  
tion
```

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 sing
 0.0556

()
 *
 0.0556-

Sen-
 sors
 of-
 ten
 re-
 qui-
 re
 ca-
 li-
 bra-
 tion
 to
 gi-
 ve
 pro-
 per
 re-
 sul-
 ts:
 -

Bias
 is
 a
 con-
 stant
 er-
 ror
 term

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 sing
 0.0556sing

()
 *
 0.0556-

By
 mea-
 su-
 ring
 a
 se-
 ries
 of
 kno-
 wn
 quan-
 ti-
 ties,
 we
 can
 com-
 pen-
 sa-
 te
 for
 the-
 se
 is-
 sues
 (ho-
 we-
 ver,
 ex-
 ter-

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 sing
 0.0556sing

()
 *
 0.0556###
 Er-
 ror
 pro-
 pa-
 ga-
 tion
 -

Cal-
 cu-
 la-
 tions
 can
 chan-
 ge
 the
 er-
 ror
 of
 a
 mea-
 su-
 re-
 ment,
 as
 the
 er-
 rors
 are
 al-

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 sing
 0.0556sing

()
 *
 0.0556###
 Ex-
 ter-
 nal
 na-
 ta
 com-
 mu-
 ni-
 ca-
 tion
 -

Sen-
 sors
 da-
 ta
 may
 need
 to
 be
 shared
 with
 ex-
 ter-
 nal
 par-
 ties,
 such
 as
 other

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 0.0556sing

()
 *
 0.0556####
 How
 to
 share
 sen-
 sors'
 da-
 ta?
 -

So-
 me
 pro-
 ces-
 ses
 (e.g.,
 an
 elec-
 tri-
 ci-
 ty
 me-
 ter,
 a
 room
 ther-
 mo-
 stat)
 42
 do
 not
 need

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 0.0556sing

()
 *
 0.0556####
 So-
 me
 so-
 lu-
 tions
 to
 send
 sen-
 sors'
 da-
 ta
 mo-
 re
 ef-
 fi-
 cien-
 tly
 -

In-
 ter-
 po-
 la-
 tion
 -

43
 In-
 ter-
 po-
 la-

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 sing
 0.0556

()
 *
 0.0556-

**Spa-
 tial
 cor-
 re-
 la-
 tion**
 -

Da-
 ta
 are
 usual-
 ly
 cor-
 re-
 la-
 ted
 in
 spa-
 ce:
 -

Traf-
 fic
 on
 ei-
 ther
 si-
 de

()
 *
 #
 Da-
 ta
 pla-
 ne
 func-
 tions
 ##
 Da-
 ta
 ac-
 qui-
 si-
 tion
 and
 pro-
 ces-
 sing
 0.0556sing

()
 *
 0.0556-

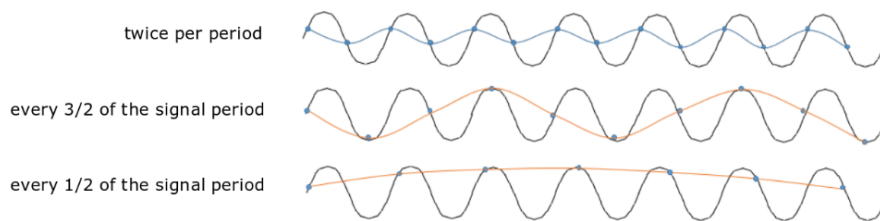
**Tem-
 po-
 ral
 cor-
 re-
 la-
 tion
 -**

Da-
 ta
 are
 al-
 so
 cor-
 re-
 la-
 ted
 in
 ti-
 me:
 -

Traf-
 fic
 in
 one
 spot
 -

7.1.4 Local control

- Provided in systems where latency is critical and/or edge autonomy is required.
- Local control enables the edge node to execute pre-defined control sequences when a specified condition occurs on some combination of local data.
- If This Then That: scenes where lights and heating/cooling are adjusted accordingly when users are leaving or approaching their homes.
- **Node-RED**: provides a graphical user interface to construct data flows with the functional processing modules and actuation output stages.
- In the extreme case of temperature, the script triggers the device shutdown.



7.1.5 Data storage

- Data storage for the acquired data may be provided by the gateway.
- The existence of local storage provides the flexibility to conserve network bandwidth and to reduce the load on cloud resources by selectively reporting data that meet certain conditions.
 - **Semantic communication.**
 - Depending on the nature of the physical thing being measured, **only some of the data points may be of interest.** E.g., successive minor changes may be of little interest, but those that exceed a comfort guard band or change by a significant percentage or amount from prior readings are.
 - Qualifying readings may be defined as events and reported only when they occur, with raw readings stored in the local database should they be required for auditing or subsequent analysis.

7.1.6 (Edge) analytics

- Edge analytics for the data may be provided by the gateway or the fog node.
- Proximity to data sources: low latency.
- Ability to process high-frequency data samples (at the rate of the sensor) → real-time.
- Conservation of network bandwidth by not sending data to the cloud.
- Can operate even in disconnected mode.
- Analytics generally require AI/ML models:
 - Insufficient computational resources at the edge.
 - No data aggregation of edge/local data.
- IoT systems should be implemented using design tools and practices that facilitate flexible allocation of functions (in tandem between edge and cloud).

7.1.7 Analytics placement

- Optimal placement of data and processing functions is an age-old trade-off in distributed systems.
 - It is based on whether it is more cost-effective to move the data to the computation or to move the computation to the data, depending on:
 - Availability and cost of bandwidth
 - Latency requirements for time-critical operations
 - Local autonomy of operations, including disconnected mode
 - Security and data control or privacy concerns
 - Cost and complexity of managing distributed nodes with computing and storage
 - Available energy resources
 - Available computational capacity
 - ...
-

Capitolo 8

Control plane functions

- Fully functional gateways may be fairly sophisticated computational and communications nodes that host and execute a variety of services.
- They need to be properly secured and managed.
- Main control functionalities:
 - Fault management (troubleshooting, error logging, and recovery).
 - Remote monitoring, control, administration, and diagnostics.
 - Remote firmware and software updates.
 - Security updates.
 - Metering (network bandwidth and software usage) and supervision.
 - Provisioning and authentication.

8.0.1 (Cyber)security

- The first step is to determine the security risk.
- Identify, quantify, and prioritize the risks for the system, and find a balance between costs and potential losses from residual risks.
- Model to quantify risk: vulnerability, threats, and impacts.



- **Vulnerabilities**: can originate from the design of the system architecture, technology, and protocols, but more often emerge in the implementation (e.g., coding flaws) and in the operation phases (e.g., configuration errors, use of weak credential, unparched devices, unexpert use, etc.).
- **Security measures for the diligent user**:

- Partitioning the system into zones (characterized in terms of security levels).
 - Adopt effective authentication protocols, tailored to the different zones.
 - Provide cryptographic protection of data.
 - Install equipment from trusted providers.
 - Make and install frequent patches and workarounds (software updates).
 - ...
- **Threat:** an IoT network is a possible point of attack if the endpoint is unguarded and is positioned in an easily reachable position.
 - **Eavesdropping:** learning confidential information.
 - **Traffic analysis:** learning data origin, destination, and length (not the content).
 - **Forgery:** building a fake message, pretending it was sent by someone else.
 - **Masquerade:** claiming to be someone else in a single message.
 - **Repudiation:** denying having sent or received a message.
 - **Profiling:** gathering information about a single user.
 - **Fingerprinting:** identifying the user associated with a message.
 - **Jamming:** causing interference to a communication.
 - **Standards:** IoT-oriented and industry-specific standards have been developed for security.

Field \ Scope	General Systems		Industrial control systems	Electric utilities	Smart Grids	Road vehicles	IoT
Service Providers	NIST Cyber Security Framework	ISO/IEC 27001	ISA/IEC 62443	ISO/IEC 27019	NISTIR 7628	ISO/SAE 21434	ISO/IEC 27400
Service Developers							ETSI EN 303 645
Device Developers							ISO/IEC 15408 Common Criteria
Specific Technologies				IEC 62351			

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

()
*

Pro-
vi-
sio-
ning
and
au-
then-
ti-
ca-
tion
-

**Pro-
vi-
sio-
ning:**
the
li-
fe-
cy-
cle
of
an
end-
point
be-
gins
with
its
pro-
vi-
sio-
ning
in
the
net-
work.
-

It
is
the
pro-
cess
of
gi-
ving
the
end-
point
the

Capitolo 9

Energy constraints

9.1 Introduction

- Most sensors and components use electrical energy, i.e., they exploit the movements of electrons across a conducting medium.
- Powering is a very important issue, and depends on the endpoint capabilities and the network architecture procedures.

0.505000 Examples
 0.505000 Files
 / bu-
 mas Smartphones Smart
 nglid
 ly de-
 re-vi-
 char
 ged
 0.505000 Battery
 st
 sen-
 sors

() ()
 * *

0.500000 Examples

() ()
 * *

0.500000 Data

li-deo-
 neca-
 me-
 ras

() ()
 * *

0.500000 Investing

vi-
 ron-
 men-
 tal
 sen-
 sors

()
 *
 ##
 Sour-
 ces
 of
 con-
 sump-
 tion
 ###
 Com-
 pu-
 0.0556ting

()
 *
 ##
 Sour-
 ces
 of
 con-
 sump-
 tion
 ###
 Com-
 pu-
 0.0556ting

()
 *
 0.0556###
 Boards
 and
 PLCs
 -

Boards
 and
 PLCs
 are
 stan-
 dard
 cir-
 cui-
 ts
 used
 in
 in-
 du-
 strial
 au-
 700-
 ma-
 tion.
 lar-

```
()
*
    ##
    Sour-
    ces
    of
    con-
    sump-
    tion
    ###
    Com-
    pu-
    0.0556ting
```

```
()
*
0.0556Example:
```

Arduino- UNO

The
Ar-
dui-
no
Uno
is
a
mi-
cro-
con-
trol-
ler
board
ba-
sed
on
the
AT-
me-
ga328.
It
con-
tains
eve-
ry-
thing
nee-
ded
to
sup-
port
the
mi-
cro-
con-

9.1.1 Communication

- Digital communication usually requires more steps than just sending a wave, and each consumes energy:
1. **Encoding and error protection.**
 - Use known redundant codes: the simplest example is the parity check bit.
 - Decoders are often complex, but can correct errors in transmission and recover the original packet. However, complexity is paid for in energy.
 2. **Preambles for synchronization and channel estimation.**
 3. **Modulation** from the baseband digital signal to the passband analog signal.
 4. **Transmission** (power is applied to the antenna) (not required for reception).
 - IoT transmissions are usually slow (low bitrate), but distance plays a large role.

()

*

0.0556##

Sour-

ce

of

ener-

gy

###

Mains

po-

wer

-

Pro-

vi-

de

vir-

tual-

ly

un-

li-

mi-

ted

po-

wer,

ma-

king

it

pos-

si-

ble

to

em-

ploy

com-

plex

pro-

to-

cols,

work

at

hi-

gh

bi-

tra-

tes,

and

74 achieve

ve

long-

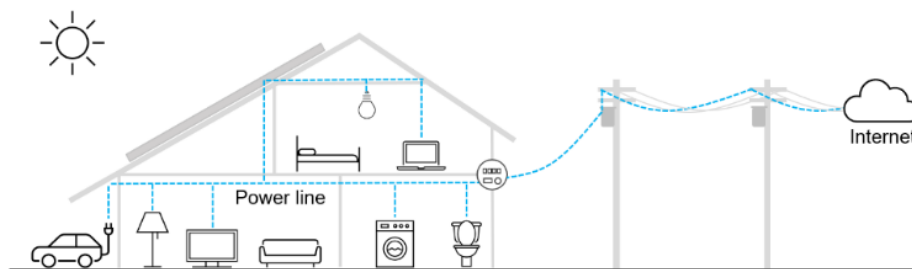
distance

9.1.2 Power over Ethernet (PoE)

- Employ the wired communication infrastructure also for powering.
 - Can carry DC power on the twisted pair used for Ethernet connectivity.
 - Only one cabled infrastructure needs to be in place.
 - Transformers are not needed.
-

9.1.3 Powerline communication

- It is the opposite of PoE.
- Use the powerline to carry data.
- Based on modulation between high-frequency data and low-frequency electricity.
- Only one cabled infrastructure needs to be in place.
- Advantageous when the system to be monitored and controlled is the power grid itself.



()
*

Bat-
te-
ries
-

Bat-
te-
ries
use
the
char-
ge
dif-
fe-
ren-
ce
bet-
ween
ions
to
in-
du-
ce
a
po-
ten-
tial
dif-
fe-
ren-
ce.
-

**Re-
char-
gea-
ble**
bat-
te-
ries
can
run
the
cir-
cuit
bac-
k-
wards
and
mo-
ve

()
*

Bat-
te-
ries
-

Bat-
te-
ries
use
the
char-
ge
dif-
fe-
ren-
ce
bet-
ween
ions
to
in-
du-
ce
a
po-
ten-
tial
dif-
fe-
ren-
ce.
-

**Re-
char-
gea-
ble**
bat-
te-
ries
can
run
the
cir-
cuit
bac-
k-
wards
and
mo-
ve

9.1.4 Solar and wind energy

- **Solar panels** are now cheap, and solar energy is renewable and clean.
 - The sensor needs to be exposed (shade can be a problem, underground or indoor sensors will not work).
 - Photovoltaic cells will not generate any power at night (the sensor needs to get through the night on a single charge).
- Another possibility is wind energy, but there are several challenges:
 - The sensor still needs to be exposed (underground or indoor sensors will not work).
 - Wind is intermittent and unpredictable.
 - Small turbines are inefficient.

()
*

Ther-
mal
and
vi-
bra-
tion
ener-
gy
-

**Ther-
mal
ener-
gy**
is
cheap
when
the
sen-
sor
has
ac-
cess
to
tem-
pe-
ra-
tu-
re
gra-
dien-
ts:
-

The
in-
si-
de/outside
or
ground/air
gra-
dient
can
ge-
ne-
ra-
te
ener-
gy.
-

()
*

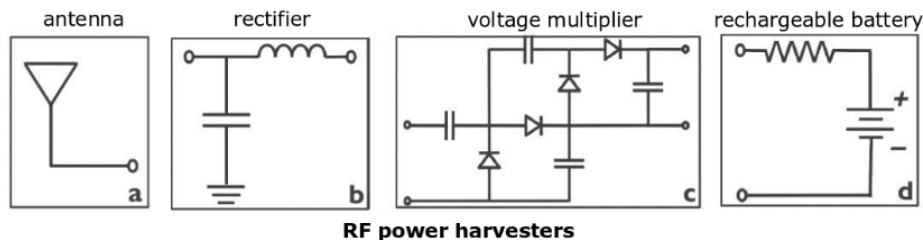
Ther-
mal
and
vi-
bra-
tion
ener-
gy
-

**Ther-
mal
ener-
gy**
is
cheap
when
the
sen-
sor
has
ac-
cess
to
tem-
pe-
ra-
tu-
re
gra-
dien-
ts:
-

The
in-
si-
de/outside
or
ground/air
gra-
dient
can
ge-
ne-
ra-
te
ener-
gy.
-

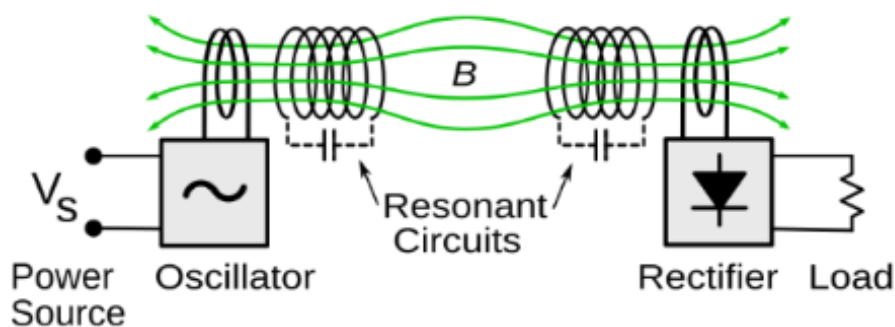
9.1.5 Passive sensors

- We can also use the received transmission to power the circuit directly: **RFID tags, biometric passports, and contactless cards** work this way.
- Power loss is huge: these only work over short distances.
- The sensor cannot actively transmit or record data, only respond to messages.
- There are **privacy issues** (anyone can read a tag, the only limit is the SNR).



9.1.6 Spectrum

- Spectrum is a limited resource (interference, collision, etc.).
- Necessary measures to ensure efficient and fair use of this resource.
- Controlled and owned by the State, that is responsible to provide geography-based regulations for the use of the spectrum.



- **Licensed spectrum**: allocation of spectrum resources to private companies (e.g., cellular network operators) upon concession fees (auctions).
 - In Italy, 5G spectrum auction in 2018: 6.5 B€ <https://www.corrierecomunicazioni.it/telco/5g/chiusa-lasta-5g-incasso-oltre-i-65-miliardi/>
 - It provides the best quality of service for end customers.
- **Unlicensed spectrum**: free access to spectrum without any centralized control.
 - *Unlicensed does not mean unregulated.*
- **“Reserved” spectrum**: for public safety, emergency forces, police, military, ...

()
*
0.0556####
Li-
cen-
sed
spec-
trum
(exam-
ples)

()

*

0.0556-

Spec-

trum

al-

lo-

ca-

tion

in

the

US

(from

300

MHz

to

3

GHz):

[https:](https://www.ntia.doc.gov/files/ntia/publications/2003-allochrt.pdf)

[//](https://www.ntia.doc.gov/files/ntia/publications/2003-allochrt.pdf)

[www.](https://www.ntia.doc.gov/files/ntia/publications/2003-allochrt.pdf)

[ntia.](https://www.ntia.doc.gov/files/ntia/publications/2003-allochrt.pdf)

[doc.](https://www.ntia.doc.gov/files/ntia/publications/2003-allochrt.pdf)

[gov/](https://www.ntia.doc.gov/files/ntia/publications/2003-allochrt.pdf)

[files/](https://www.ntia.doc.gov/files/ntia/publications/2003-allochrt.pdf)

[ntia/](https://www.ntia.doc.gov/files/ntia/publications/2003-allochrt.pdf)

[publications/](https://www.ntia.doc.gov/files/ntia/publications/2003-allochrt.pdf)

[2003-allochrt.](https://www.ntia.doc.gov/files/ntia/publications/2003-allochrt.pdf)

[pdf](https://www.ntia.doc.gov/files/ntia/publications/2003-allochrt.pdf)

-

4G

LTE

LI-

CEN-

SED

fre-

quen-

cy

bands

in

Ita-

ly:

[https:](https://www.sqimway.com/site_band.php)

[//](https://www.sqimway.com/site_band.php)

[www.](https://www.sqimway.com/site_band.php)

[sqimway.](https://www.sqimway.com/site_band.php)

[com/](https://www.sqimway.com/site_band.php)

[site_](https://www.sqimway.com/site_band.php)

[band.](https://www.sqimway.com/site_band.php)

[php](https://www.sqimway.com/site_band.php)

Unlicensed spectrum

- Also referred to as **Instrumental, Scientific, and Medical (ISM)** bands.
 - It is available to any number of customers, but with no exclusive rights.
 - **Target:** applications which are not lucrative enough to justify the cost of licence, but prove valuable if considered collectively.
 - Notable examples: **Wi-Fi, Bluetooth**, etc.
 - **Unlicensed does not mean unregulated** → there are still some rules to avoid/minimize interference, and to promote fair use of resources.
 - Some regulation mechanisms: **ERP limits, LBT, duty cycle**, etc.
 - Regulations are not necessarily aligned in all the Regions of the world, but there are some levels of superimposition (e.g., **2.4 GHz ISM band** for Wi-Fi).
-

Unlicensed spectrum in Europe: ERP

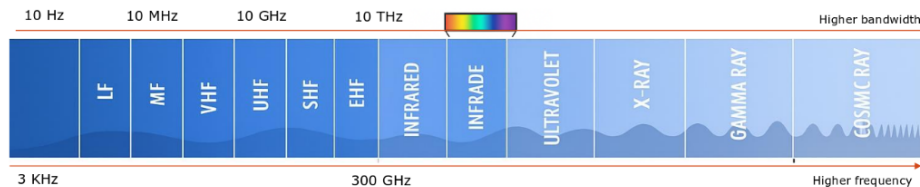
- At the European level, spectrum is regulated by the **European Commission (EC)**, **Electronic Communications Committee (ECC)**, and the **European Telecommunications Standard Institute (ETSI)**.
 - Different rules/limitations are defined.
 - **Effective Radiated Power (ERP)**: it evaluates the emissions of a device.
 - It is related to the **Effective Isotropic Radiated Power (EIRP)**, defined as the power that would have to be used on an isotropic antenna in order to get the same field strength that the tested device produces at the same distance.
 - We have that **ERP = EIRP – 2.15**.
-

Unlicensed spectrum in Europe: LBT

- **Listen Before Talk (LBT)**: a radio transmitter first senses its radio environment before it starts a transmission.
 - The device continuously monitors channels to transmit only when a channel is not in use.
 - For a device to transmit its data (talk), it needs to ensure that the channel is free (listen).
 - **How to monitor/sense the channel?**
 - **Energy detection:** The receiver measures the received power in the band of interest over a sensing interval. If the measured energy exceeds a pre-defined threshold, the channel is considered busy.
 - **Layer-2 frame decoding:** The receiver tries to detect and decode known structures in the frame. If a valid preamble or header is detected, the channel is declared busy.
-

Unlicensed spectrum in Europe: DC

- **Duty Cycle (DC):** it is defined as the ratio, expressed as a percentage, of the maximum transmitter “on” time over one hour, relative to a one hour period.
- These limitations apply to every receiver, excluding those with LBT capabilities.
- **1% DC:** the max. transmission time of any device is 1% of an hour (i.e., 36 seconds in an hour).



Unlicensed spectrum in Europe

- Some of the most important ISM sub-bands in Europe (focus on IoT use cases).
- Country specific.

```

() () () () ()
* * * * *
Range Duty Example
0.1103 0.1176 Example
0.2255 IoT
0.2255 IoT sce-
na-
0.3162 rio

() () () () ()
* * * * *
0.1003 0.1054 0.1075 0.1096 0.1117 Wa-
169.875 Meter
/ 0.1% and
500 M-gas
mW burst-
(169.875-
169.875) 15g
() () () () ()
* * * * *
0.1003 0.1054 0.1075 0.1096 0.1117 Me-
434.4170 NRC-
ring

```

```

() () () () ()
* * * * *
      Ram Duty Example
      0.1178 Example
0.1103 [Mhz] - IoT
      0.2055 sce-
      na-
      0.3162 rio

```

```

() () () () ()
* * * * *
0.1003 0.8802 0.5670 Me-
870.1 Wr Rate-
/ 0.124g,
500 be Smart
m 10% ci-
(8696.4,
8696.5) ==
vi-
ron-
men-
tal
con-
trol ==

```

```

() () () () ()
* * * * *
0.1003 0.8802 0.5670 Smart
2483.5 to ch,
Wime ==
Fi ==

```

```

() () () () ()
* * * * *
0.1003 0.8802 0.5670 Smart
5350.1 Fisho-
si (802.1)
po-
wer
con-
trol

```

```

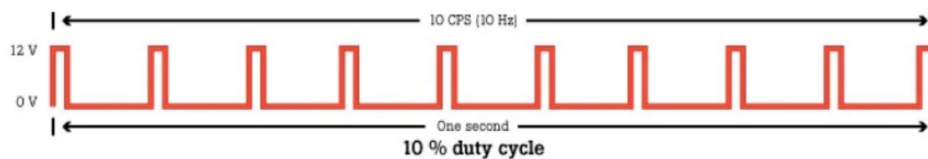
() () () () ()
* * * * *
0.1003 0.8802 0.5670 Smart
5725.1 Fisho-
si (802.1)
po-
wer
con-
trol

```

9.1.7 Short-range

Introduction

- **Short-range:** (generally) cell diameter smaller than 100 m.
- Does not support mobility/handover.
- Can be classified according to the reach of the radio link:
 - **Vicinity and proximity networks**, from a few cm to ~2–3 m.
 - **Wireless Personal Area Networks (WPANs)**, up to ~10 m.
 - **Wireless Local Area Networks (WLANs)**, up to ~100 m.
- Are sometimes referred to as **Low-Rate WPAN**.



()

*

0.0556#

Ra-

dio

Fre-

quen-

cy:

IDen-

ti-

fi-

ca-

tion

(RFID)

and

NFC

()

*

0.0556##

RFID

(
 *
 0.0556####
 Over-
 view
 Ra-
 dio
 Fre-
 quen-
 cy
 Iden-
 ti-
 fi-
 ca-
 tion
 (RFID)
 is
 a
 me-
 thod
 for
 re-
 mo-
 te-
 ly
 sto-
 ring
 and
 re-
 trie-
 ving
 da-
 ta
 th-
 rou-
 gh
RFID
tags
 and
rea-
ders.
 Tags
 sto-
 re
 in-
 for-
 ma-
 tion
 that
 90 readers
 can
 ac-

9.1.8 How It Works

RFID tags have embedded chips storing information that can be rewritten. When a reader emits a radio signal, the **tag's coil** couples inductively with the reader's coil, transferring energy (a form of **wireless power transfer**).

The tag uses this energy to respond, enabling data exchange over variable distances.

<https://youtu.be/-Wf7aadxBkE?si=e7GMOLnFuQ8he80f>



9.1.9 Near-field Coupling

The coupling between tag and reader happens in the **near-field region**, meaning the distance must be much smaller than the signal wavelength ($\approx 15\%$ of it).

Operating at lower frequencies increases the wavelength, thus expanding the working distance:

- At **2.4 GHz**, $\lambda \approx 0.13$ m $\rightarrow$ range ≈ 20 cm
- At **13.56 MHz**, $\lambda \approx 22$ m $\rightarrow$ range ≈ 3 m

9.1.10 Tags Visualized

Missing Image: Pasted image 20251014084153.png

RFID tags typically include an **antenna** and an **integrated circuit (chip)**.

The antenna captures electromagnetic energy from the reader, while the chip stores and processes information.

Different tag designs exist depending on frequency range, application, and power type (active, semi-passive, passive).

9.1.11 Applications

RFID systems are widely used for:

- **Security:** identification cards, passports

- **Access control:** Telepass, metro cards

- **Retail and logistics:** tracking merchandise, inventory, pallet management

- **Ticketing and sports:** electronic ticket systems, race tracking

- **NFC in smartphones:** contactless interactions

Readers can monitor entries/exits or track specific tagged items for process monitoring.

9.1.12 Applications: Passport

Since 2006, **RFID tags** have been embedded in **British and U.S. passports**.

They store printed information (including a **digital photograph**) and consist of: - A **microchip** containing personal data

- A **tag antenna** for communication

- A **substrate** to hold components together

These **biometric passports** allow for faster and more secure identity verification.

9.1.13 Applications: Metro Card

Metro systems use **RFID cards** to manage balances securely through cryptographic protection.

When a passenger enters or exits a station, the reader updates the card balance locally: - **Entry**: records time and station

- **Exit**: calculates fare and updates balance

Readers periodically sync with a central server for auditing, lost card tracking, and balance verification.

Users can also check balances online.

9.1.14 Applications: Merchandise in Stores (1/3)

RFID tags in retail enable: - **Data storage**: more information than barcodes, editable anytime

- **Quality control**: tracking goods from production to storage

- **Speed**: read multiple tags simultaneously at a distance

- **Inventory management**: automatic product location and stock updates

- **Durability**: resistant to handling and environmental stress

Common example: **clothing RFID tags**.

9.1.15 Applications: Merchandise in Stores (2/3)

RFID supports **anti-theft** and **authenticity verification**.

When an unpaid item passes through security gates: - The **transmitter's** radio waves induce a current in the tag's antenna. - The tag emits a secondary signal at a specific frequency. - The **receiver** detects this and triggers an alarm.

Note: Security gates are tall to enhance detection field coverage.

(
 *
 0.0556####
 Ap-
 pli-
 ca-
 tions:
 Mer-
 chan-
 di-
 se
 in
 Sto-
 res
 (3/3)
 At
 chec-
 kout,
RFID
tags
are
deac-
ti-
va-
ted
 to
 pre-
 vent
 theft
 or
 fal-
 se
 alarms.
 A
deac-
ti-
va-
tion
pad
 di-
 sa-
 bles
 the
 elec-
 tro-
 nic
 com-
 po-
 nents
 in
 the

9.1.16 RFID vs. Barcode

Feature	RFID	Barcode
Rate	Thousands simultaneously	One at a time
Range	Up to a few meters	~6 meters
Speed	Very fast (milliseconds)	Slow
Line-of-sight	Not required	Required
Identification	Unique per item	Item type only

RFID is thus superior for automation, scalability, and inventory tracking.

9.1.17 Electronic Product Code (EPC)

The **EPCGlobal** consortium (founded in 2003) standardized RFID operation.

The **Electronic Product Code (EPC)** uniquely identifies any physical object across the world.

Key aspects: - Encodes product and manufacturer info. - Accessible through **EPC Information Services (EPCIS)** via **web services** (SOAP, WSDL). - Enables global traceability for logistics and retail applications.

()

*

0.0556####

Elec-

tro-

nic

Pro-

duct

Co-

de

(EPC)

Struc-

tu-

re

The

EPC

is

com-

po-

sed

of

se-

ve-

ral

fields:

-

Hea-

der

(8

bi-

ts):

de-

fi-

nes

the

par-

ti-

tio-

ning

sche-

me

(up

to

256

in-

stan-

ces)

(
 *
 0.0556This
 struc-
 tu-
 re
 gua-
 ran-
 tees
 a
**glo-
 bal-
 ly
 uni-
 que
 ID**
 for
 eve-
 ry
 item,
 across
 all
 ma-
 nu-
 fac-
 tu-
 rers
 and
 ti-
 me.

9.1.18 Types of Tag

RFID tags differ by **power source** and **functionality**: - **Semi-passive**: contain a small battery activated by reader signals; typical lifespan 3–5 years. Used in temperature or environmental sensors.

- **Active**: have a dedicated battery for continuous transmission, allowing **longer range** and **richer data** (e.g., Telepass, asset tracking, personnel tags).

Applications include **cold-chain logistics**, **warehouse monitoring**, and **vehicle tracking**.

9.1.19 Active Tag: Telepass

Telepass uses **Active RFID** at around **5.8 GHz** (microwave frequency).

When the vehicle enters a toll gate: 1. Roadside sensors emit a microwave signal to wake nearby devices.

2. The in-car transponder activates and sends its **unique ID**.

3. The system verifies the ID in the database and opens the barrier if authorized.

Active RFID ensures **fast, reliable, and long-distance** communication.

()
*
0.0556####
Ty-
pes
of
Tag
(Pas-
si-
ve)
**Pas-
si-
ve
RFID
tags**
ha-
ve
no
in-
ter-
nal
bat-
te-
ry
and
re-
ly
on
ener-
gy
tran-
smit-
ted
by
the
rea-
der.
They
are
ex-
tre-
me-
ly
**low-
cost**
and
used
in
101
most
RFID
ap-

()
 *
 0.0556They
 pro-
 vi-
 de
 short
 com-
 mu-
 ni-
 ca-
 tion
 ran-
 ges
 but
 are
 ideal
 for
 ac-
 cess
 con-
 trol,
 re-
 tail,
 and
 tic-
 ke-
 ting
 sy-
 stems.

9.1.20 Passive Tag Types

Two main standards define range and operation: - **Vicinity (ISO/IEC 15693)**: range up to 1–1.5 m

- **Proximity (ISO/IEC 14443)**: range up to 2–10 cm

Features: - Strong resistance to interference and unwanted readings

- Commonly used in **ticketing**, **contactless payments**, and **access control**

Transmission includes a **bitstream** and a **CRC** for data integrity checking.

9.1.21 Passive Tag (PHY Layer)

Since passive tags are simple, they use **amplitude-based modulation**: - **Amplitude Shift Keying (ASK)**: carrier amplitude varies to represent bits

- High amplitude → bit 1
- Low amplitude → bit 0

This scheme allows the tag to decode data while harvesting energy from the same signal.

Missing Image: Pasted image 20251014085031.png

()

*

0.0556####

Pas-

si-

ve

Tag

(PHY

Layer)

—

On-

Off

Key-

ing

(OOK)

On-**Off****Key-****ing****(OOK)**

is

a

sim-

pli-

fied

ver-

sion

of

ASK

with

on-

ly

two

am-

pli-

tu-

de

le-

vels:

-

“ON”

(full

am-

pli-

tu-

de)

105-

pre-

sen-

ts

()

*

0.0556Ad-

van-

ta-

ges:

ve-

ry

sim-

ple

im-

ple-

men-

ta-

tion.

Dra-

w-

backs:

-

Lo-

wer

da-

ta

ra-

te

than

ASK

(on-

ly

2

le-

vels)

-

The

tag's

po-

wer

sup-

ply

de-

pends

on

si-

gnal

am-

pli-

tu-

106; de;

long

se-

quen-

$$\frac{()}{*} \\ 0.0556$$

9.1.22 Passive Tag (PHY Layer) — Pulse Interval Encoding (PIE)

Passive tag communication must balance data transmission and power delivery.

Pulse-Interval Encoding (PIE) keeps the signal at full power most of the time: - A **short power-off pulse** after a long power interval → bit 1

- A **shorter power interval** after the same off pulse → bit 0

This ensures the tag receives energy even during long streams of 0s.

As a result, **data rate depends on bit patterns** (0s transmit faster than 1s).

Missing Image: Pasted image 20251014085939.png

9.1.23 Passive Tag (MAC Layer)

MAC design is challenging due to potentially **thousands of tags** and limited tag intelligence.

- If one reader → no need for complex MAC.

- Multiple readers → interference must be managed.

Tags cannot use advanced techniques like carrier sensing or RTS/CTS.

Two major anti-collision protocols: 1. **Slotted ALOHA**: tags transmit after random back-off times.

2. **Binary Tree Resolution**: the reader queries subsets of tags by partial ID patterns until only one tag responds.

9.2 Bluetooth

9.2.1 Overview

Bluetooth is a **wireless technology** designed for connecting devices like telephones, computers, cameras, and printers over short distances.

- Initially proposed by **Ericsson** and standardized as **IEEE 802.15.1**.

- Created to replace wired connections within computers.

- Three types of Bluetooth:

- **Bluetooth Classic (BR/EDR)** - **Bluetooth High Speed** - **Bluetooth Low Energy (BLE)**

9.3 Bluetooth

9.3.1 Applications

- **Bluetooth Classic (BR/EDR):**
 - **Audio streaming** (headphones, speakers, car audio systems).
 - **Peripheral devices** (keyboards, mice, printers).
 - **File transfers** (sending files between phones, computers, and other devices).
 - **Bluetooth High Speed:**
 - **High-resolution video streaming.**
 - **Tethering** for internet sharing between devices.
 - **Bluetooth Low Energy:**
 - **Wearables.**
 - **Beacons** for proximity marketing and location-based services.
 - **Smart home devices.**
-

9.4 BR/EDR

9.4.1 Topology

- Devices connect in small networks called **piconets** (2–8 active devices, up to 255 parked devices).
- **Master-Slave architecture:** One device acts as the **Master**, and others act as **Slaves**.
- Master controls the network with a **polling algorithm**.
- Multiple piconets can combine to form a **scatternet**.

Missing Image: Pasted image 20251014093810.png

Frame The **frame structure** includes: - **Access code:** Synchronization and piconet ID.

- **Header:** Contains an 18-bit pattern repeated three times with information such as address (3 bits for 7 destinations), message type, flow control (F), acknowledgment (A), sequence number (S), and header error correction (HEC).

- **Payload:** Carries data, ranging from 0–2740 bits.

Missing Image: Pasted image 20251014093827.png

()
*

Layers
Blue-
tooth
com-
mu-
ni-
ca-
tion
fol-
lo-
ws
a
laye-
red
ar-
chi-
tec-
tu-
re:
-

The
PHY
layer
deals
with
the
phy-
si-
cal
tran-
smis-
sion
of
da-
ta.
-

The
MAC
layer
hand-
les
ac-
cess
con-
trol
and
de-
fi-
nes

()
*

Layers
Blue-
tooth
com-
mu-
ni-
ca-
tion
fol-
lo-
ws
a
laye-
red
ar-
chi-
tec-
tu-
re:
-

The
PHY
layer
deals
with
the
phy-
si-
cal
tran-
smis-
sion
of
da-
ta.
-

The
MAC
layer
hand-
les
ac-
cess
con-
trol
and
de-
fi-
nes

9.4.2 FHSS Coordination

- Bluetooth devices must **synchronize** their frequency hopping sequences during **link establishment**.
- **Frequency hopping** helps reduce interference, ensuring smooth communication between devices.

9.4.3 Baseband Layer

- The **Baseband layer** is responsible for organizing the communication between devices, similar to the **MAC layer** in LANs.
- Bluetooth uses **Time Division Duplex (TDD)** with 625 μ s slots.
- Communication alternates between the **Master** (even slots) and **Slave** (odd slots), with different frequency channels used in each slot.

Missing Image: Pasted image 20251014094743.png

()

*

0.0556####

Con-

nec-

tion

Pro-

ce-

du-

re

1.

In-

qui-

ry

Pha-

se:

-

Ma-

ster

ini-

tia-

li-

zes

com-

mu-

ni-

ca-

tion

and

esta-

bli-

shes

the

hop-

ping

se-

quen-

ce

for

the

pi-

co-

net.

-

De-

vi-

114es

go

in-

to

()

*

0.0556Col-

li-

sion

bet-

ween

two

sla-

ves

is

ve-

ry

ra-

re

to

hap-

pen.

If

it

does,

he

sim-

ply

ask

for

a

re-

tra-

smis-

sion.

Odd

and

even

di-

stinc-

tion

is

do-

ne

be-

cau-

se

ma-

ster

can-

not

tra-

smit

115 and

re-

cei-

vo

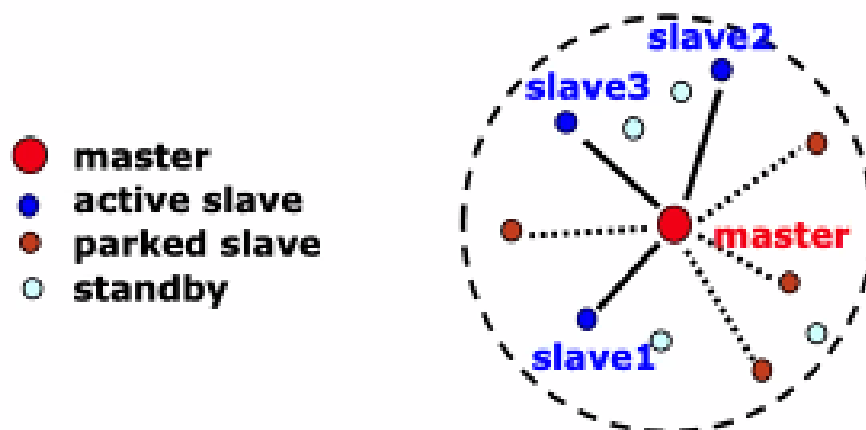
9.4.4 L2CAP

- The **Logical Link Control and Adaptation Protocol (L2CAP)** handles **multiplexing, segmentation, reassembly, QoS (Quality of Service), and group management**.
- Two types of communication:
 - **Synchronous Connection-Oriented (SCO)**: Low-latency, connection-oriented service.
 - **Asynchronous Connection-Less (ACL)**: Packet-oriented, error-checked, and asynchronous.
 - If a payload encapsulated in the frame is corrupted, it is retransmitted.

9.5 BR/EDR

9.5.1 L2CAP ACL Example

- **L2CAP ACL** packets can be 1, 3, or 5 slots long, with each slot occupying a different frequency channel.
- This reduces overhead by minimizing the need for extra headers and guard times, making data transmission more efficient.



9.5.2 LMP (Link Manager Protocol)

- The **Link Manager Protocol (LMP)** is responsible for **setting up and managing Baseband connections**.
- LMP handles authentication, link configuration, and **power management**.
- It exchanges **Protocol Data Units (PDUs)** between two paired devices to configure settings such as pairing, authentication, and packet size optimization.

()
*

HCI
(Ho-
st
Con-
trol-
ler
In-
ter-
fa-
ce)
-

The
**Ho-
st
Con-
trol-
ler
In-
ter-
fa-
ce
(HCI)**
pro-
vi-
des
a
com-
mand
in-
ter-
fa-
ce
bet-
ween
the
**Ho-
st**
(up-
per
layers)
and
**Con-
trol-
ler**
(Ba-
se-
band,
LMP,
PHY).

()
*

HCl
(Ho-
st
Con-
trol-
ler
In-
ter-
fa-
ce)
-

The
**Ho-
st
Con-
trol-
ler
In-
ter-
fa-
ce
(HCl)**
pro-
vi-
des
a
com-
mand
in-
ter-
fa-
ce
bet-
ween
the
**Ho-
st**
(up-
per
layers)
and
**Con-
trol-
ler**
(Ba-
se-
band,
LMP,
PHY).

9.6 BLE

9.6.1 Overview

- **Bluetooth Low Energy (BLE)** is a more recent extension of Bluetooth, designed specifically for **low-power IoT devices** and sensors.
 - Key differences from BR/EDR:
 - **Fewer channels:** 40 (2 MHz) vs. 79 (1 MHz).
 - Uses **GFSK modulation** only.
 - Data rate of up to **2 Mbit/s**, which results in **shorter transmission times** and **lower energy consumption**.
 - **Adaptive frequency hopping**, with potentially fewer hops.
 - New error protection codes for more reliable communication.
-

9.7 BLE

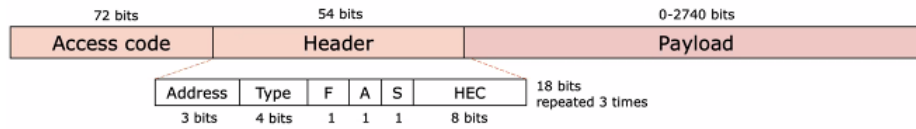
9.7.1 Communication Modes

- BLE nodes can take on **4 roles**, as opposed to just **2 in BR/EDR**:
 - **Broadcaster:** Sends advertisements but does not accept connections (e.g., iBeacon).
 - **Observer:** Listens for advertisements but does not initiate connections.
 - **Peripheral:** Sends advertisements and may accept connections, acting as a **Slave**.
 - **Central:** Can initiate connections and acts as the **Master** after receiving information about a peripheral.
-

9.8 BLE

9.8.1 New Profiles

- **New BLE profiles** support specialized applications, such as:
 - **Heart Rate Profile**, **Internet Protocol Support Profile**, and the **Mesh Profile**.
- **Mesh Profile** enables **multi-hop communication** in **mesh networks**:
 - **Relay nodes** forward messages between devices.
 - **Low Power Nodes (LPN)** sleep and wake periodically to exchange data with **Friend Nodes**, which store messages while the LPN is asleep.
 - **Managed flooding** prevents loops in the network.



()
 *
 0.0556##
 BR/EDR
 vs. BLE
 ###
 Com-
 pa-
 ri-
 son
 The
 fol-
 lo-
 wing
 ta-
 ble
 com-
 pa-
 res
 the
 fea-
 tu-
 res
 of
BR/EDR
 and
BLE:

()

*

0.0556###

Key

Dif-

fe-

ren-

ces

-

BR/EDR

is

ideal

for

ap-

pli-

ca-

tions

re-

qui-

ring

hi-

gh

da-

ta

ra-

tes

and

low

la-

ten-

cy,

such

as

au-

dio

and

vi-

deo

strea-

ming,

whi-

le

BLE

is

op-

ti-

mi-

zed

128
for

low-

power

do-

9.8.2 General description

Overview

- IEEE 800.15.4 defines the **Physical (PHY)** and **Media Access Control (MAC)** sublayers for Low-Rate Wireless Personal Area Networks (LR-WPANs).
- It is designed for easy installation, reliability, extremely low cost, reasonable battery life, and relaxed throughput.
- It supports a large number of nodes (unlike BLE) and asynchronous communication of small data packets.
- It is the foundation used by protocols like **ZigBee** and **6LoWPAN**.

Operational frequency

Band	868.3 MHz	915 MHz	2.4 GHz
Channels	1	10	16
Bandwidth/channel	600 KHz	2 MHz	2 MHz
Modulation	BPSK	BPSK	QPSK
Bitrate	~20 Kbit/s	~40 Kbit/s	~250 Kbit/s
Sensitivity		-92 dBm	-85 dBm
Transmission power		<1 mW (low-power)	
Availability	Europe	America	Worldwide
Range		10-75 m (typically 30 m)	
Interference		Few	Severe (very crowded)

Types of nodes

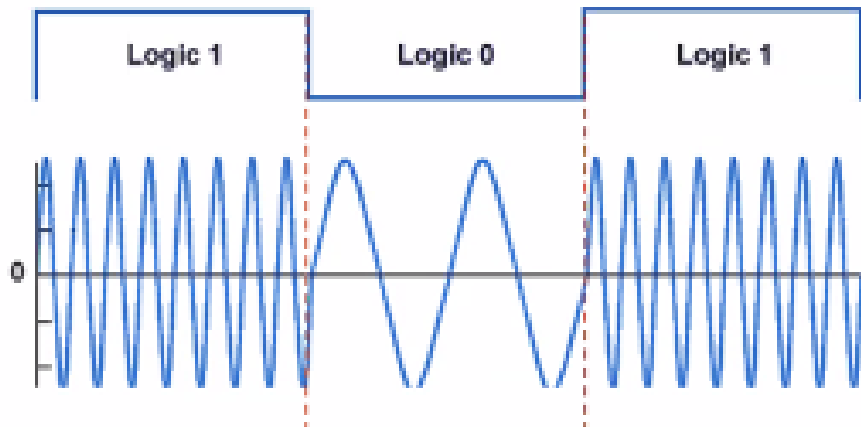
- **Full Function Device (FFD):**
 - Has full PHY and MAC capabilities.
 - Can relay messages.
 - Can act as a PAN coordinator.
 - Supports any topology.
- **Reduced Function Device (RFD):**
 - Simpler implementation with reduced functions.
 - Can only talk to an FFD and cannot be a PAN coordinator.
 - Cannot relay data (acts only as an end node).
 - Limited to star topology.

Topologies

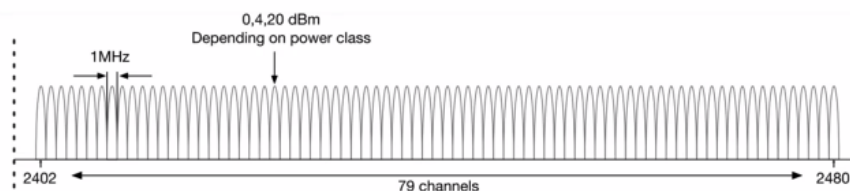
- Star topology
- Mesh topology
- Cluster tree topology more masters than BLE

Frame structure

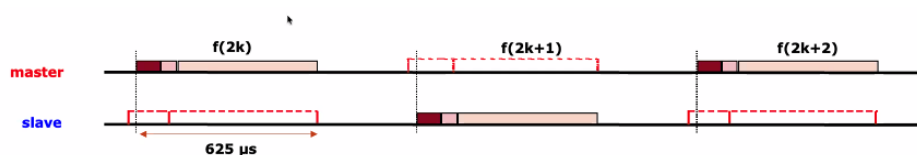
- **Preamble (32 bits):** For synchronization.



- Start of Packet Delimiter (8 bits).
- PHY Header (8 bits): Indicates PSDU length.
- PSDU (0 to 1016 bits / 0-127 bytes): The data field.

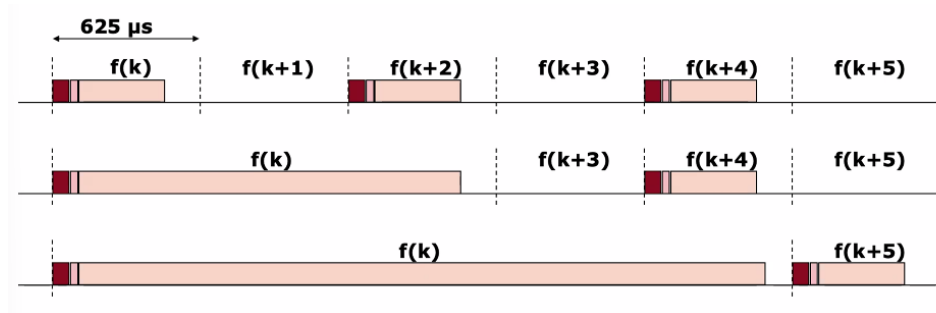


MAC layer



The MAC layer operates in two main modes:

- **Beacon-enabled mode:**
 - The PAN coordinator periodically sends **beacons** for synchronization.
 - Time is divided into **Superframes** (the active part).
 - The Superframe is divided into 16 time slots and has two parts:
 1. **Contention Access Period (CAP):** Uses slotted CSMA/CA for channel access.
 2. **Contention Free Protocol (CFP) (optional):** Uses **Guaranteed Time Slots (GTS)** allocated by the coordinator for dedicated, contention-free resources.
 - An **Inactive part** allows nodes to sleep to save power.
 - The ratio of superframe duration to beacon interval is the **duty cycle**.
- **Non-beacon-enabled mode:**
 - No periodic beacons are sent.
 - Channel access uses **unslotted CSMA/CA**.



- Synchronization is not required, which saves power from beacon listening.
- Devices must use **active scanning** (sending a Beacon Request) to join the network.

Security

The MAC layer provides several security services: * **Unsecured:** No security. * **Access Control List (ACL):** Accepts/rejects frames based on the source node. * **Secured:** Implements multiple mechanisms: * Access control. * Data confidentiality (symmetric encryption). * Frame integrity. * Sequential freshness protection (prevents frame replay).

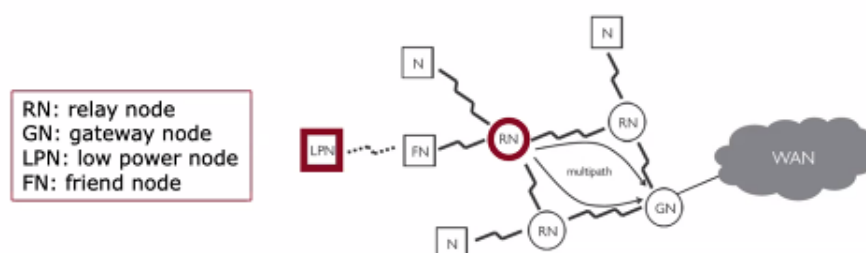
Amendments

- **Timeslotted Channel Hopping (TSCH):** Designed for industrial processes. It works in non-beacon mode and uses channel hopping to improve reliability.
- **Deterministic and Synchronous Multi-channel Extension (DSME):** Implements the use of multiple channels within the same superframe for robustness.

9.8.3 ZigBee

Overview

- An industrial standard for IoT promoted by the Zigbee Alliance.
- It is built on top of the IEEE 800.15.4 PHY and MAC layers.
- It defines the **Network (NWK)**, **Application Support (APS)**, and **Application Layers (AF)**.



Applications

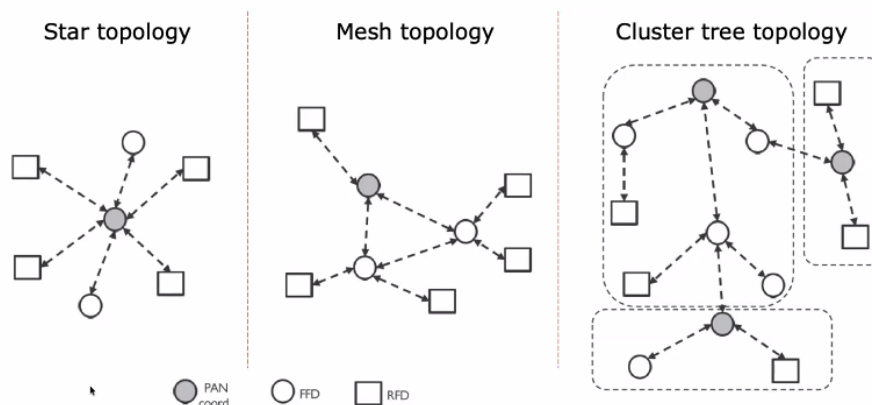
- Building automation (HVAC, lighting, metering)
- Personal healthcare (patient monitoring)
- Industrial automation (asset management, process control)
- Consumer electronics (remotes)
- PC peripherals (mouse, keyboard)

Network layer: functionalities

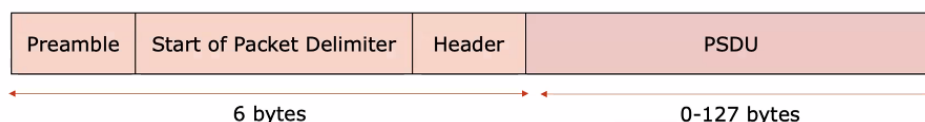
- Starting, joining, and leaving a network.
- Configuring new devices.
- Assigning addresses to devices.
- Routing and route discovery.
- Applying and removing security to frames.

Network layer: topology

- **Star Network:** A central coordinator (FDD) communicates with end devices (FDD or RFD).



- **Mesh Network:** The network is extended using **ZigBee routers (FDDs)** that relay data, allowing nodes to communicate beyond the coordinator's direct range.
- ##### Network layer: addressing Each device has two identifiers:



1. **Extended Address (64-bit):** A unique, universal IEEE address assigned during production (like a MAC address).
2. **Short Address (16-bit):** A dynamic ID assigned by the PAN Coordinator when a device joins the network (like an IP address). This allows for up to ~65,000 nodes.

Network layer: routing

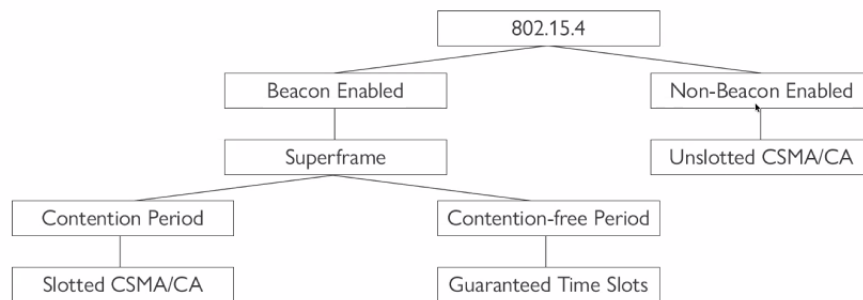
- Can be **hierarchical** (following the network tree).
- Also uses **dynamic “best” path** searching based on the **AODV (Ad-hoc On-demand Distance Vector)** protocol.
- Routers are discovered on-demand.
- The algorithm finds the minimal cost path, calculated from the probability of packet delivery on each link.

$$C\{l\} = \min(7, [\frac{1}{p_l^4}])$$

where p_l is the probability of a packet delivery on link l . It depends on the quality of the link.

Application layer: overview

Consists of three sublayers: 1. **Application Support (APS)** 2. **ZigBee Device Objects (ZDO)** 3. **Application Framework (AF)** (manufacturer-defined apps)



Application layer: APS

Provides services such as: * **Address mapping** (between 64-bit and 16-bit addresses). * **Fragmentation and reassembly**. * **Security** (encryption/decryption). * **Binding** (matching two devices based on their services). * **Service Discovery**. * **Group Addressing** (e.g., “turn on all lights”).

Application layer: ZDO

A special application that handles key network functions: * Defining the device’s role (Coordinator, Router, End Device). * Device discovery and network formation. * Network management (joining/leaving). * Security management (key distribution). * Network address management (resolving conflicts).

Application layer: AF

- Provides the platform for running the actual user application (Application Objects).

- Provides application **profiles** to ensure interoperability between different manufacturers.
- Exposes ZDO functions to the applications.

9.8.4 6LoWPAN

Overview

- **6LoWPAN** is an IETF standard for **IPv6 over Low-Rate Wireless Personal Area Networks** (i.e., IEEE 800.15.4).
- The goal is to transform any device into an IPv6-addressable host.
- It introduces a **6LoWPAN Adaptation layer** between the IPv6 Network layer and the 800.15.4 MAC layer.

IPv6

- **Problem:** IPv4 has up to 4B addresses.
- We may need in the future a larger address space.
- How to solve?
 - Classless addressing
 - Private addressing and NAT.
 - **IPv6**
- IPv6 addresses are written in **hexadecimal** separated by “:”.
- Three types: **Unicast, Multicast, Anycast**, Broadcast **not** supported.
- It can have multiple addresses: **Link-Local, Unique-Local, Global**

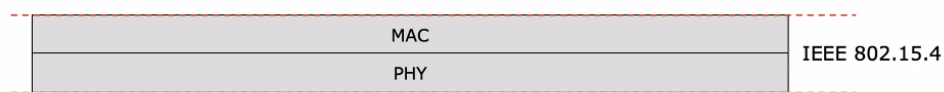
Link-Local Address

- **Prefix:** Starts with **FE80::/10** (the first 10 bits are equal to 1111 1110 10).
- **Purpose:** Can be used to reach nodes attached to the same link without a globally unique address. If several IPv6-enabled nodes connect to a switch, they will **auto-configure** their interfaces with link-local addresses, discover each other, and be able to communicate.
- **Key Features:**
 - Contains the MAC address of the node in the EUI-64 format.
 - **Not routable:** IPv6 routers must not forward packets having link-local source/destination address.
 - All IPv6-enabled interfaces have a link-local unicast address.
- **Structure:** 10 bits for the prefix, 54 reserved bits (zeros), and 64 bits for the Interface ID.

B	Contention Access Period (CAP)										GTS 1		GTS 2			
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	

Unique Local Address (ULA)

- **Prefix:** Starts with a prefix $\text{FC00::}/7$ (the first 7 bits are equal to 1111 110).
- **Purpose:** It is a **private** IPv6 address used for internal communication within an organization or network, similar to private IPv4 addresses.
- **Key Features:**
 - **Not routable** to the public Internet; won't overlap with any other ISP-assigned range.
 - It allows sites to be interconnected without creating any address conflicts.
 - It supports flexible **subnetting** and routing internally within the private network.
- **Structure:** 7 bits for the prefix, 1 bit for 'L', 40 bits for the Global Routing Prefix (randomly generated number ensuring global uniqueness), 16 bits for the Subnet ID, and 64 bits for the Interface ID (typically the MAC address in EUI-64 format).



Global Unicast Address

- **Prefix:** Starts with a prefix $2000::/3$ (the first 3 bits are equal to 100).
- **Purpose:** **Public address:** it can be used to route IP datagrams over the Internet.
- **Key Features:**
 - The structure consists of a 48-bit prefix and a 16-bit subnet ID also referred to as **Site-Level Aggregator (SLA)**.
 - The prefix is variable, defined from router advertisements, and some IP addresses can be reserved.
- **Structure:** 3 bits for the prefix, 45 bits for the Global Routing Prefix, 16 bits for the Subnet ID, and 64 bits for the Interface ID.
 - The first 64 bits make up the **Network ID (NetID) Prefix**.
 - The last 64 bits make up the **HostID Suffix**.



IPv6 assignment

1. Manual config

2. Stateful config with **DHCPv6** protocol
 3. Stateless autoconfig without DHCP, IPv6 nodes can connect to a network and automatically generate a Global IPv6 address with no dedicated server. #####
Digression: IPv6 for IoT
- **Benefits** of using IPv6 protocols in IoT scenarios:
 - Can connect potentially infinite number of IoT.
 - Easily connect to other devices without the need of translation.
 - Address/manage/access any IoT device from the Internet (no saturation).
 - Use well-known socket API for the deployment of the network applications.
 - Easily re-use tools for managing, commissioning, and diagnosing IP-based networks.
 - **Problem:** There is a large mismatch between IPv6 and IEEE 800.15.4.
 - **MTU Size:** The minimum IPv6 MTU is **1280 bytes**.
 - **800.15.4 MTU:** The maximum 800.15.4 frame payload is **127 bytes**.
 - **Header Overhead:** The standard IPv6 header is **40 bytes**, which would consume nearly one-third of the entire 800.15.4 payload.
-

General architecture

- 6LoWPAN networks connect to the internet using standard **IPv6 edge routers**.
- This avoids the need for special application-layer gateways (like those used in ZigBee).
- Network nodes can be **Hosts (H)** (end devices) or **Routers (R)** (FFDs). #####

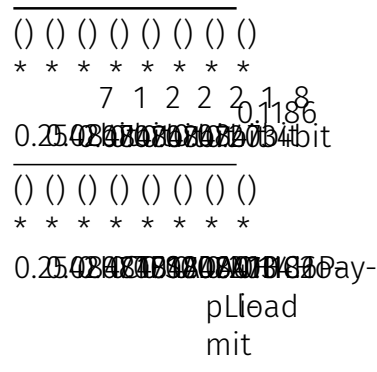


Addressing To solve the MTU and overhead problem, 6LoWPAN uses:

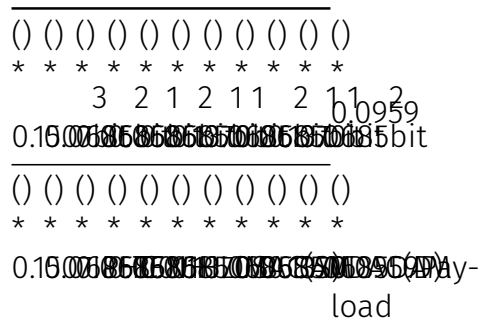
1. **Header compression**
2. **Fragmentation**

Header compression

- Works by eliding or compressing redundant information from the 40-byte IPv6 header.
- Fields like Version (always 6), Payload Length (derived from data-link layer), and Traffic Class/Flow Label (often zero) can be compressed or removed. ##### HC1 format:



IPHC format A more efficient format that uses context-based compression.



SAM/DAM value	Inline	IID
000	128	address fully inline
001	64	link-local 64-bit Interface ID
010	16	link-local 16-bit short address
011	0	link-local + link-layer source address
100		reserved
101	64	context + 64-bit Interface ID
110	16	context + 16-bit short address
111	0	context

Fragmentation

- If a packet is still too large after compression, it must be fragmented.
- Fragmentation information is carried in a **Fragment Header**.

First Fragment

8 bit	11 bit	16 bit
11000xxx	Size	Tag

Subsequent Fragments

8 bit	11 bit	16 bit	8 bit
11100xxx	Size	Tag	Offset

Routing

- Standard IPv6 routing is too resource-consuming for IoT nodes.
- 6LoWPAN requires a new routing scheme, the de-facto standard being **RPL (Routing Protocol for Low Power and Lossy Networks)**.

Routing (RPL)

- **Topology:** RPL proactively (not on-demand) builds a **Destination-Oriented Directed Acyclic Graph (DODAG)**. This is a directed graph without cycles, oriented towards a root node (the edge router).
- **Rank:** Each node is assigned a **rank**, which represents the cost to reach the root. The rank must increase as you move down the graph (away from the root). This mechanism is used to **avoid loops**.
- **Control Packets:** RPL uses control packets to build and maintain the DODAG:
 - **DIO (DODAG Information Object):** Sent downwards (from root to leaves) to establish the upward path.
 - **DAO (Destination Advertisement Object):** Sent upwards (from leaves to root) to establish the downward path.
 - **DIS (DODAG Information Solicitation):** Used by nodes to request a DIO message.
- **Modes of Operation:**
 1. **Storing:** Intermediate nodes store routing tables for all destinations in their sub-DODAG. DAOs are sent to the parent.
 2. **Non-Storing:** Only the root (edge router) stores routing information. All routing passes through the root. DAOs are sent all the way to the root.
- **DODAG Creation Example:**
 1. The Edge Router (ER, Rank 0) multicasts a **DIO** message.
 2. Receiving nodes (e.g., 1, 2, 3) respond with a unicast **DAO** message sent back up the path.
 3. The parent/root sends a **DAO-ACK** to acknowledge.
 4. Nodes 1, 2, and 3 compute their own rank (e.g., Rank 1) and rebroadcast the **DIO** message to their neighbors (e.g., 4, 5, 6).
 5. If a node (e.g., 4) receives multiple DIOs, it selects a **preferred parent** based on the lowest rank.
 6. The process continues until all leaf nodes are reached.

Capitolo 10

Internet of Things and Smart Cities - 07 - Sigfox

10.1 Title and Authorship

This document is a part of the “Internet of Things and Smart Cities” course, specifically on the topic of “07 - Sigfox” or “IoT technologies (long-range).”

10.2 Network Architectures

The presentation reviews two main network architectures:

- **One-level architecture:** A system where a device (represented by a car) communicates directly over a Wide Area Network (WAN) to a distributed systems interconnect, which then connects to the Intranet/Internet.
- **Two-level architecture:** This includes an intermediate network level.
 - **Scenario 1:** A device (represented by a trash can) communicates over a Low-Power Wide Area Network (LPWAN) to a **gateway**, which then connects to the WAN and the distributed systems interconnect.
 - **Scenario 2:** A device/system (represented by a factory) communicates over a Local Area Network (LAN), which connects via a **router/gateway** to the WAN and the distributed systems interconnect.

In both architectures, the distributed systems interconnect acts as a central point, connecting to the Intranet/Internet for access by other devices (e.g., mobile phones, computers).

Capitolo 11

Long-range IoT Technologies: Introduction

11.1 Long-Range Communication

“Long-range” generally refers to a cell diameter of **several kilometers**. This is typically enabled by cellular networks, which come with trade-offs: * **High complexity** and **infrastructure cost**. * Support for **very high data rates**. * **Not designed for energy efficiency** (recharging mobile terminals is normal).

11.2 Low-Power Wide Area Networks (LPWAN)

LPWANs are a category of technologies designed to **jointly optimize** a combination of energy efficiency, data rate, communication range, and cost per device.

The following table compares LPWAN to other wireless network types:

() () () ()
 * * * * *

WSN Cellular
 0.1479 Parameter LPWAN
 (Wired-
 0.2113 works
 less
 Sen-
 sor
 Net-
 0.2113 work)

() () () ()
 * * * * *

0.1079 Range Low-
 ry (~ ~ ry
 sh 101 long
 (~100 (~
 1-m) km)
 3 20
 m) km)

() () () ()
 * * * * *

0.1079 Energy Low-
 ef-ry di (tx
 fi-hi (tx low
 cion pow (tx.
 cy (tx ver po-
 po ~ 500 ver
 we 80mW)
 < mW) 20
 1 mW)
 mW)

() () () ()
 * * * * *

0.1079 Range Low-
 ra- ry ghry
 te hi- low
 gh

() () () ()
 * * * * *

0.1079 Consumption Low-
 di dion
 low

11.3 LPWAN Features and Examples

LPWAN technologies, which include **Sigfox**, **LoRa**, and **NB-IoT** (the latter being cellular-based), share key characteristics: * **Long-range coverage** (up to 20 km, sometimes higher). * **Very high energy efficiency**, allowing battery life up to 10 years. * **Low communication/infrastructure cost**. * **Low data rate** per device, but support for a **very large number of devices**. * **Easy network implementation and deployment** (no need for detailed radio planning). * **Mostly static** with limited support for handover and mobility due to a simple core network.

Capitolo 12

Sigfox

12.1 Overview

Sigfox is the first LPWAN technology, developed by the French company Sigfox. The core idea is to provide a Sigfox cloud platform through a subscription-based model.

The system architecture works as follows: 1. **End Nodes** (devices) communicate via the **Sigfox Radio** link to **Sigfox Base Transceiver Stations (BTSS)**. This communication is managed by local operators. 2. The BTSSs forward the data to the **Sigfox Cloud** using a backhaul connection (e.g., 4G, xDSL, IP) for storage and processing. 3. The **Sigfox Cloud** is composed of: * **Backend Servers**: Handle BTS management, monitoring, and message processing. * **Storage**: Stores modems metadata and messages. * **Frontend Servers**: Manage users/groups and modems. 4. Data stored and processed in the Sigfox Cloud can be accessed in two primary ways: * **Web-Interface & Application Integration**: Users can access the data via a standard **Web browser** using HTTPS. * **Customer Application Server**: Customer servers can integrate with the Cloud using HTTPS, REST/JSON, or Email to receive data, API responses, alerts, and call backs.

12.2 Coverage

Sigfox has achieved significant global coverage. The presentation includes a map illustrating: * **Live coverage** (shown in one color, generally a darker shade like purple or magenta). * **Country under roll-out** (shown in a lighter color, generally blue or cyan).

The map shows coverage in parts of North America, South America, Europe, Asia, and Australia, highlighting a continuous rollout process across different regions.

Capitolo 13

Sigfox Applications and Frequency Range

13.1 Applications (Page 10)

Sigfox is well-suited for **long-range large-scale IoT applications** where the data rate requirements are limited. Examples include: * **Connected dumpers** (e.g., OnePlus Systems, Sayme). * **Street lightning** (e.g., Kawantech). * **Smart parking** (e.g., IoTMalta, Libelium, Sterela). * **Gas tank remote monitoring** (e.g., Silicon Controls, Ijinus).

13.2 Frequency Range: Radio Configurations (RCs) (Page 11-12)

Sigfox operates across different frequency bands globally, categorized into Radio Configurations (RCs):

- The presentation includes a map showing the geographical distribution of various RCs (**RC1** through **RC7**).
- The frequency bands, data rates, and other specifics vary per RC:

*	*	*	*	*	*	*	*
0.1050	0.1050	0.1050	0.1050	0.1050	0.1050	0.1050	0.1050
*	*	*	*	*	*	*	*
0.1050	0.1050	0.1050	0.1050	0.1050	0.1050	0.1050	0.1050
fre-							
quen-							
cy							
(MHz)							

```
( ) ( ) ( ) ( ) ( ) ( ) ( )
* * * * * * * *

0.1250 0.2500 0.3750 0.5000 0.6250 0.7500 0.8750 C7
-----
( ) ( ) ( ) ( ) ( ) ( ) ( )
* * * * * * * *

0.1250 0.2500 0.3750 0.5000 0.6250 0.7500 0.8750 1.0000
fre-
quen-
cy
(MHz)

( ) ( ) ( ) ( ) ( ) ( ) ( )
* * * * * * * *

0.1250 0.2500 0.5000 0.7500 1.0000 1.2500 1.5000 1.6000
da-
ta
ra-
te
(bit/s)

( ) ( ) ( ) ( ) ( ) ( ) ( )
* * * * * * * *

0.1250 0.2500 0.5000 0.7500 1.0000 1.2500 1.5000 1.6000
da-
ta
ra-
te
(bit/s)

( ) ( ) ( ) ( ) ( ) ( ) ( )
* * * * * * * *

0.1250 0.2500 0.5000 0.7500 1.0000 1.2500 1.5000 1.6000
(dBm)

( ) ( ) ( ) ( ) ( ) ( ) ( )
* * * * * * * *
```

- **RC1** is associated with a 1% Duty Cycle (DC), and **RC2** and **RC4** use Frequency Hopping (FH).

Capitolo 14

Sigfox Physical (PHY) Layer

14.1 Simplicity and Ultra-Narrow Band (UNB)

The core keyword for the Sigfox PHY layer is **simplicity**: there is no connection, no configuration, and no signaling. * Sigfox uses a proprietary **Ultra-Narrow Band (UNB)** wireless technology. * Energy is highly concentrated into a very tiny portion of the bandwidth. * This approach results in a **very limited data rate**. * A key benefit is its **interference-immune** nature, as the UNB signal has a small bandwidth and is less impacted by conventional, wider-band interferers.

14.2 Modulation and Repetition

- **Modulation:** Sigfox uses Differential Binary Phase Shift Keying (**DBPSK**) and Gaussian Frequency-Shift Keying (**GFSK**).
- In **DBPSK**, bits are encoded as changes in the signal's phase. A data bit '0' means the phase is not reversed, while a data bit '1' means the phase is reversed.
- **Repetition Scheme:** Each message is transmitted **3 times** (triplication). This provides more robustness by making it likely that at least one Sigfox station receives the signal. The Sigfox cloud/backend is responsible for handling and removing possible duplicate messages.

14.3 Data Rate and Bandwidth

- In DBPSK, each symbol carries 1 bit of information (since '0' is no phase change and '1' is phase reversal).
- The symbol rate is generally less than or equal to the channel bandwidth.
- Therefore, in a typical **100 Hz-wide channel**, the maximum practical symbol rate is about 100 baud, which translates to **100 bit/s** for DBPSK (1 bit per symbol).

Capitolo 15

Sigfox Frame Structure and MAC Layer

15.1 Frame Structure

The maximum frame size in Sigfox is **26 bytes**. The structure differs slightly between uplink (UL) and downlink (DL) frames:

15.1.1 Uplink (UL) Frame

- **Payload:** Up to **12 Bytes**.
- UL transmission is more common than DL (e.g., for sensors to report data measurements).
- **Structure:** Preamble (4 bytes), Sync. (2 bytes), Device ID (4 bytes), Payload (variable $0 \div 12$ bytes), Auth. (variable, not specified in size), FCS (2 bytes).

15.1.2 Downlink (DL) Frame

- **Payload:** Up to **8 Bytes**.
- **Structure:** Preamble (8 bytes), Sync. (13 bits), Flags (2 bits), FCS (1 byte), Auth. (2 bytes), Error codes (variable, not specified in size), Payload (variable $0 \div 8$ bytes).

15.2 Transmission Times and Limits

The system has very strict transmission limits, exemplified by the Sigfox Europe (RC1) configuration:

- **Duty Cycle (DC):** 1%. This means a device can transmit for a total of **36 seconds per hour**.
- **UL Data Rate:** 100 bit/s.
- **Hourly Limit (Raw Bits):** With the DC, a device can transmit 3600 bits/h, which equals **450 bytes/h**.
- **Limit for Unique Data:** Due to the repetition code (3 times), the limit for “new” data is $450/3 = 150$ bytes/h.

- **Frame Limit:** With a 26-byte frame size, a device can send approximately $450/(26 \cdot 3) \simeq 6$ frames/h*.
- **Max UL Payload Limit:** This translates to a maximum useful payload of 12 bytes
· 6 frames/h = 72 bytes/h, which is very limited.

This limited capacity is why Sigfox is only suitable for applications with limited data requirements, such as: * GPS coordinates (lat x long): 6 bytes * Temperature: 2 bytes
* State reporting: 1 byte

15.3 MAC Layer

- **Transmission Initiation:** Transmission is always initiated by the end devices.
- **Protocol:** Sigfox uses **Unslotted ALOHA**, characterized by:
 - No channel sensing.
 - No synchronization.
 - No continuous connection.
- **Cycle:** End devices **periodically wake up** to transmit UL data, briefly listen for any downlink messages (like ACKs or commands), and then return to **sleep mode**.
- **Use Case:** This method is effective for regular data collection and monitoring but is **not ideal for command-and-control** applications, as any command must wait for a sensor-initiated transmission window.

Capitolo 16

Internet of Things and Smart Cities - Long Range (LoRa)

16.1 Overview

This document covers LoRa technology, which stands for Long Range. LoRa was developed in France in 2009, and the LoRa Alliance was founded in 2015.

LoRa and LoRaWAN define different parts of the protocol stack: * **LoRa**: The Physical (PHY) layer implementation. * **LoRaWAN**: The rest of the protocol stack, specifically the MAC layer and higher layers (Application Layer).

The protocol stack layers are: Application Layer, MAC Layer, PHY Layer, and Radio Frequency Layer.

16.2 Coverage

The presentation includes a map illustrating the coverage of networks built on the LoRaWAN protocol. The map shows numerous deployed gateways across the globe, with clusters in regions like Europe and North America. An inset image highlights a specific region, showing multiple individual gateways and providing details for one of them, including its name, ID, EUI, network server, last heard timestamp, and coordinates.

Capitolo 17

LoRa Applications and Frequency Range

17.1 Applications

LoRa is used for various large-scale IoT applications, including: * **Smart agriculture:** Used for cattle management (e.g., tracking) and monitoring the health of soil. * **Smart cities:** Used for city networks, monitoring bus schedules, and other urban infrastructure. * **Smart buildings:** Used for reading and collecting utility usage data. * **Smart environment:** Used for fire monitoring systems.

17.2 Frequency Range

LoRaWAN uses regional specifications to establish rules for various parameters, including preamble, channel frequencies, allowed spreading factors (SF), maximum payload size, receive windows, and join procedures.

Key frequency bands used in different regions are:

Region	Frequency (MHz)
EU (Europe)	863-870 (commonly 868 MHz)
US (United States)	902-928
China	779-787 and 470-510

17.3 Frequency Range (Europe)

The European frequency band (EU863-870) has specific regulations for power and duty cycle: * The band employs between **24 and 80 channels**, each 125 KHz wide. * The network operator determines the number of channels used, with at least 3 mandatory channels for join-requests: 868.10, 868.30, and 868.50 MHz. * The maximum MAC payload size is **230 bytes** at the best data rate.

Specific band regulations in Europe include:

Capitolo 18

LoRa Physical (PHY) Layer

18.1 Modulation

- **Modulation Type:** LoRa utilizes **Chirp Spread Spectrum (CSS)**.
- **Mechanism:** Symbols are encoded by modulating a carrier that **changes frequency linearly in time** (chirps). This spreads the signal power across a wide region of the spectrum to resist frequency-selective noise.
- **Chirps:** The modulation uses both up-chirps and down-chirps, which are confined within the designated bandwidth.
- **Symbol Structure:** Each LoRa symbol corresponds to one complete chirp waveform, where the frequency increases linearly over the symbol duration, causing the amplitude peaks to get closer together.

18.2 Spreading Factor

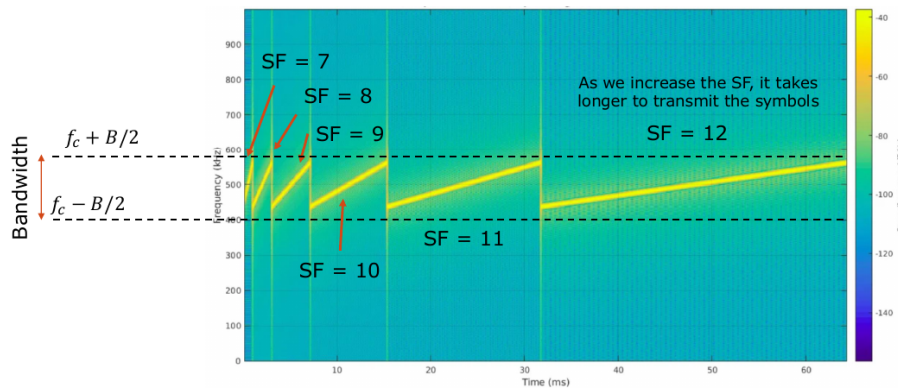
- Number of chips used to encode a symbol
- Each symbol is made of 2^{SF} and each symbol represents **SF bits** of information.
 - Example: SF = 2 → 2 bits to represent symbols → 2²=4 possible symbols: {00,01,10,11}.
- The value 2^{SF} defines the number of discrete time steps (chips) to make up one chirp. Therefore, it is the resolution of the waveform.
- SFs influence data rate, time-on-air, battery life, and receiver sensitivity.
- In LoRa, SF can assume values between 7 and 12.

18.2.1 Spreading Factor, Data Rate, and Time

The Spreading Factor (SF) is critical in determining transmission characteristics:

- * **Symbol Duration (T_S):** The time to transmit a symbol is calculated as $T_S = \frac{2^{SF}}{B}$, where B is the bandwidth.
- * **Symbol Rate (R_S):** The number of symbols per second is $R_S = \frac{B}{2^{SF}}$.
- * **Bit Rate (R):** The number of bits per second is calculated as $R = SF \cdot R_S = \frac{SF \cdot B}{2^{SF}}$.
- * Increasing the SF causes an exponential increase in symbol duration, which is the dominant factor, leading to a **decrease** in the overall bit rate. The energy consumption also increases as the symbol duration (Time on Air) increases.

Spreading Factor



Orthogonality * **Pseudo-Orthogonal SFs**: LoRa SF values are pseudo-orthogonal. This allows multiple signals using different data rates (i.e., different SFs) to be transmitted on the same channel and received simultaneously by the gateway without causing a collision. * **Collision Resistance**: Signals using the same SF can still be received without collision if their power difference is ≥ 6 dB.

18.2.2 Code Rate (CR)

- **Forward Error Correction (FEC)**: LoRa employs FEC using a Code Rate (CR), which adds a number of extra bits every 4 bits, with CR ranging from 1 to 4.
- **Actual Bit Rate**: The final, actual bit rate is calculated by including the CR correction in the formula:

$$R = SF \cdot \frac{B}{2^{SF}} \cdot \left(\frac{4}{4 + CR} \right)$$

LoRa Physical (PHY) Layer: Sensitivity and Adaptive Data Rate (ADR)

18.3 Sensitivity

- **SF and Sensitivity**: Higher SF values lead to a longer transmission duration (Time on Air, ToA). This allows the receiver to “accumulate” energy for a longer time.
 - **Better Sensitivity**: The result is **better sensitivity**—the receiver is capable of effectively receiving more corrupted signals. This translates to a longer communication range because the receiver can successfully receive over a more attenuated link.
 - **Minimum SNR**: The required minimum Signal-to-Noise Ratio (SNR_m) decreases as the SF increases. For instance, SF7 requires -7.5 dB, while SF12 requires -20 dB. (transmitting for a longer amount of time, with higher consumption and lower SNR)

18.4 Data Rate and Trade-offs

The trade-off between SF and performance parameters is key: * **Higher SF** leads to a **Lower bit/data rate**, a **Higher ToA**, and a **Higher range**. * The maximum allowed

payload size at the application layer decreases as the SF increases to help reduce the ToA.

A summary of the trade-offs (for $BW = 125$ KHz and $CR = 1$) is provided:

[illegible]

18.5 Adaptive Data Rate (ADR)

- **ADR Mechanism:** The Adaptive Data Rate (ADR) is a mechanism used to select the optimal SF based on the link conditions.

- **Objective:** The goal is to jointly optimize the communication quality, network capacity, and power consumption. The SF should be large enough to ensure reach but not too large to waste energy.
- **Logic:**
 - If the link margin is high (good signal), the SF can be **reduced** (data rate increased).
 - If the link budget is low (poor signal), the SF should be **increased** (data rate reduced) to cope with the worse SNR.
- **Impact of ADR:** As SF increases with distance from the gateway, the ToA also increases, along with power consumption. The visualization shows that end devices far from the gateway (up to 14 km) use SF12 with the lowest bit rate, while those closest to the gateway use SF7 with the highest bit rate.

Capitolo 19

LoRa Physical (PHY) Layer: Frame Structure

19.1 Physical Frame Structure

The physical frame is composed of several fields, where the payload size can range from 0 to 255 bytes:

() () ()

* * *

[illegible]

() () ()

* * *

0.3333333333333333

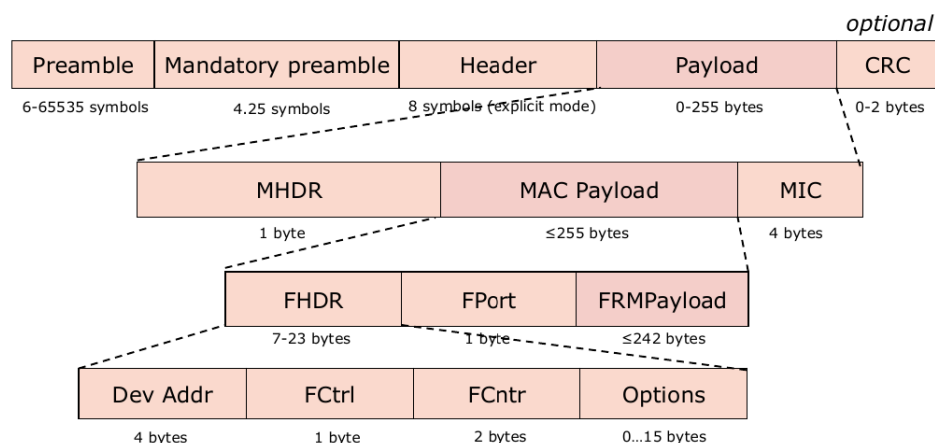
255a-
byte
test being
tran-
smit-
ted.

() () ()
* * *

0.3333333333333333-Cy-

2 clic byRe-teslun-
dan-
cy Check
in-
di-
ca-
tion
for
the
pay-
load.

19.2 MAC Payload Structure



The **Payload** field in the physical frame is actually the encrypted MAC Payload, which has its own internal structure:

1. **MHDR (MAC Header):** 1 byte. Specifies the message type (e.g., join-request, data up/down).
2. **MAC Payload:** This section has a maximum size of ≤ 255 bytes and contains the core data and control information. It is composed of:
 - **FHDR (Frame Header):** 7 – 23 bytes. Contains key addressing and control information:
 - **DevAddr:** (32 bits / 4 bytes) The device address.
 - **FCtrl (Frame Control):** 1 byte. Contains flags, such as whether to use the data rate specified by the gateway for uplink, whether the message acknowledges the reception of a previous message, or if the gateway has more data for the mote.
 - **FCntr (Frame Counter):** 2 bytes. A sequence number used for security and replay protection.
 - **Options:** 0...15 bytes. Used for MAC commands to change data rate, power transmission, etc.
 - **FPort (Frame Port):** 1 byte. Defines the application port number. It only exists if application data is present.
 - **FRMPayload (Frame Payload):** ≤ 242 bytes. This is the application data itself.
3. **MIC (Message Integrity Code):** 4 bytes. A digital signature used for security.

The minimum header of the MAC payload is 12 bytes (MHDR + FHDR + MIC), plus 1 byte for FPort if application data is sent.

Capitolo 20

LoRa Time on Air (ToA) and Transmission Examples

20.1 Time on Air (ToA) Calculation

The Time on Air (ToA) is the total time required to transmit a frame, calculated as the sum of the preamble time and the payload time: $\text{ToA} = t_{\text{preamble}} + t_{\text{payload}}$.

- **Preamble Time:** $t_{\text{preamble}} = (n_{\text{preamble}} + 4.25) \cdot T_s$, where T_s is the symbol duration.
- **Payload Time:** $t_{\text{payload}} = n_{\text{payload}} \cdot T_s$, where n_{payload} is the number of symbols in the payload.

The calculation of n_{payload} is complex and depends on many parameters: * Spreading Factor (SF). * Payload bytes (PL). * Header enabled status (H: implicit or explicit mode). * Low Data Rate Optimization enabled status (DE). * Code Rate (CR).

The full formula for n_{payload} is derived from the Semtech SX1276 datasheet:

$$n_{\text{payload}} = 8 + \max \left(\left\lfloor \frac{8\text{PL} - 4\text{SF} + 28 + 16\text{CRC} - 20\text{H}}{4(\text{SF} - 2\text{DE})} \right\rfloor \cdot (\text{CR} + 4), 0 \right)$$

The overall ToA can be expressed in terms of SF and B :

$$\text{ToA} = \frac{2^{\text{SF}}}{B} \cdot ((n_{\text{preamble}} + 4.25) + n_{\text{payload}})$$

20.2 Transmission Time Examples

The Time on Air is highly dependent on both the Spreading Factor and the total packet size. For a fixed bandwidth ($B = 125$ KHz) and a packet size of 50 bytes:

DR	SF	ToA (ms)
DR5	7	97.5 ms
DR4	8	174.6 ms
DR3	9	328.7 ms

20.2. Transmission Time Examples LoRa Time on Air (ToA) and Transmission Examples

DR	SF	ToA (ms)
DR2	10	616.4 ms
DR1	11	1,314.8 ms
DR0	12	2,302.0 ms

This visualization clearly shows that **increasing the SF dramatically increases the Time on Air**, which directly impacts how often a device can transmit due to regulatory duty cycle limits.

Capitolo 21

LoRa Transmission Times and Constraints

21.1 Duty Cycle Limitations

LoRaWAN transmissions are subject to strict regulatory constraints, particularly the duty cycle:

- **Duty Cycle Rule:** In the European region (EU868), a common maximum duty cycle is 1%. This means the transmitter must be silent for 99% of the time after a transmission. The total maximum transmission time is 36 seconds per hour.
- **Impact of ToA on Message Count:** The Time on Air (ToA) dramatically limits the number of messages that can be sent per hour:
 - **High Data Rate Example (SF7):**
 - * ToA for a 100 B packet is 189.7 ms.
 - * The required silent time is 18.78 seconds.
 - * Total cycle time is ~ 18.97 s.
 - * A device can send ~ 190 packets per hour (of 100 B).
 - **Low Data Rate Example (SF12):**
 - * ToA for a 100 B packet is 4431.9 ms (approx. 4.43 s).
 - * The required silent time is 438.7 seconds (approx. 7.3 minutes).
 - * Total cycle time is ~ 443.2 s.
 - * A device can send only ~ 8 packets per hour (of 100 B).

21.2 Fair Access Policy

In addition to the regulatory duty cycle, some network operators, such as The Things Network, implement a “Fair Access Policy” that further restricts usage:

- **Policy Limit:** This policy caps the average uplink Time on Air to **30 seconds per 24 hours per device**. This is a much stricter limit than the 1% duty cycle (~ 36 seconds per hour).
- **Consequence:** This policy effectively limits the number of messages a device can send per day to a very low number, particularly at high SFs.

A useful website: <https://avbentem.github.io/airtime-calculator/ttn/eu868/100>

The Fair Access Policy acts as a further constraint beyond the regulatory duty cycle: * **Uplink Limit:** An average of **30 seconds of uplink Time on Air per 24 hours per device**. * **Effective Duty Cycle:** This constraint is much stricter, equating to approximately 1.25 seconds/hour (compared to 36 seconds/hour for the 1% duty cycle). * **Downlink Limit:** At most **10 downlink messages per 24 hours**, which includes ACKs for confirmed uplinks. * **Message Limits under FAP:** This policy drastically reduces the number of messages a device can send per day: * High Data Rate (SF7, 100 B ToA $\approx$ 189.7 ms): $\sim$ 158 packets per day. * Low Data Rate (SF12, 100 B ToA $\approx$ 4.43 s): $\sim$ 6 packets per day.

21.3 LoRaWAN Architecture

LoRaWAN defines the components and recommended protocols for the network, which uses a **star-of-stars topology**.

The architecture involves four main components: 1. **End Nodes:** End devices that perform sensing and actuating tasks. Each is identified by a DevEUI (64 bits). 2. **Gateway (Concentrator):** Wireless base stations that receive messages from all End Nodes within range, based on the LoRa physical layer. They relay the messages to the Network Server using a backhaul link (e.g., 4G, xDSL, Ethernet). Each is identified by a DevAddr (32 bits). 3. **Network Server:** The central element that: * Eliminates duplicate messages (since End Nodes transmit messages that can be received by multiple gateways). * Routes data. * Determines the optimal data rate (using ADR). * Selects the best Gateway to reach the End Node in Downlink (DL) transmissions based on link quality. 4. **Application Server:** The final destination for application data.

Security: The system provides two layers of AES security: * **Network Session AES secured messages** between the End Node and Network Server. * **End-to-End AES secured payload** between the End Node and Application Server.

Capitolo 22

LoRaWAN End Node Classes and Channel Access

22.1 Types of End Nodes (Classes A, B, and C)

LoRaWAN defines three distinct classes of End Nodes that dictate their bi-directional communication capabilities and, consequently, their energy consumption:

22.1.1 1. Class A (Base Class)

- **Communication:** Always initiated by the End Node (UL message is completely unsolicited and asynchronous).
- **Downlink (DL):** After sending an UL message, the End Node opens two short receive windows: RX1 and RX2. The DL communication (e.g., commands or ACK) occurs during these windows. If the Network Server doesn't respond in RX1, it tries RX2. If neither responds, DL must wait for the next UL transmission.
- **Energy/Latency:** **Very low energy consumption** (most of the time in sleep mode) but **very large DL latency**.
- **Use Case:** Periodic sensor measurements.

22.1.2 2. Class B (Bi-directional Class)

- **Communication:** Operates as Class A (with RX1 and RX2) but adds time-synchronized receive windows.
- **Downlink (DL):** The Gateway sends **beacons** every 128 seconds. The End Node uses these beacons for synchronization to open dedicated **ping slots (PS)**. These ping slots, which last 30 ms each, allow the Gateway to initiate DL transmissions.
- **Energy/Latency:** **Lower (bounded) DL latency** (DL does not necessarily depend on UL transmission) but **higher energy consumption** than Class A.
- **Use Case:** Utility meters (electrical, water), street lights.

22.1.3 3. Class C (Continuous Receive Class)

- **Communication:** Operates as Class A, but the second receive window (RX2) remains **open continuously** until the next UL transmission.
- **Downlink (DL):** DL messages can be received at almost any time via RX2.
- **Energy/Latency:** **Very low DL latency** but **very high energy consumption**.
- **Constraint:** Class C devices are generally required to be connected to the mains (not battery powered).
- **Use Case:** Beacon lights, alarms.

22.2 Channel Access Mechanism

- **Mechanism:** LoRaWAN relies on a random-access scheme known as **Pure ALOHA**.
- **Operation:** Each End Device transmits whenever it has data to send, with **no coordination** among nodes.
- **Collision Mitigation:** While collisions can occur, overlapping transmissions can still be decoded because the Spreading Factors (SFs) are quasi-orthogonal.

22.3 Executive Summary

NarrowBand-IoT (NB-IoT) is a cellular technology standardized by the 3GPP, specifically engineered to address the unique requirements of the Internet of Things (IoT). It leverages existing public cellular infrastructure (4G-LTE and 5G-NR) to provide a low-power, wide-area network (LPWAN) solution. Designed for low complexity and minimal resource consumption in terms of power and spectrum, NB-IoT is positioned as a leading solution for massive IoT deployments.

The technology operates in licensed frequency bands and offers three distinct deployment modes—Standalone, In-band, and Guard-band—allowing for flexible integration into new or existing cellular networks. Key features include advanced power-saving mechanisms like Power-Saving Mode (PSM) and Extended Idle Mode DRX (eDRX), which significantly extend the battery life of endpoint devices. To ensure robust connectivity in challenging environments, NB-IoT employs a repetition mechanism, dynamically adjusting data transmissions to achieve a maximum coupling loss (MCL) of 164 dB, enabling deep indoor and wide-area coverage of up to 10 km. Data rates are optimized for typical IoT messaging, with effective throughputs reaching up to 62.5 Kbps in the uplink and 26.2 Kbps in the downlink.

22.4 1. Introduction and Core Concepts

The proliferation of IoT devices necessitates wireless solutions that differ significantly from traditional cellular networks, which are optimized for high bitrates and not power efficiency. NB-IoT was developed as a specifically designed, cellular-based solution to fill this gap, offering a more efficient alternative to adapting existing 3G or 4G protocols.

Core Design Principles:

- **Standardization:** NB-IoT is standardized by the 3rd Generation Partnership Project (3GPP), the body responsible for major mobile telecommunications standards.
- **Foundation:** It is based on the design principles of 4G-LTE, such as Orthogonal Frequency Division Multiple Access (OFDMA), but is not directly compatible, requiring its own dedicated radio access with unique control signals and channels.
- **Objectives:** The primary targets are low device complexity, minimal power consumption, and efficient use of spectrum.

22.4.1 1.1. Key Applications

NB-IoT is suitable for a wide range of applications that require long-range, low-power communication for sending small amounts of data.

- **Smart Agriculture:** Collection of soil and environmental data.
- **Healthcare Monitoring:** Real-time transmission of vital health data from remote monitoring devices.
- **Retail:** Inventory management, asset security, and enhancement of customer experiences.
- **Energy Management:** Smart meters providing real-time data on electricity consumption to improve efficiency.
- **Supply Chain and Logistics:** Asset tracking and monitoring.
- **Public Safety:** Connectivity for safety and emergency services infrastructure.

22.5 2. Technical Architecture and Specifications

22.5.1 2.1. Frequency Spectrum and Bands

NB-IoT operates in licensed frequency bands, using the same bands as LTE but typically favoring the lower range to increase signal range. The use of licensed spectrum distinguishes it from technologies like LoRa, which operate in unlicensed bands.

LTE Bands for NB-IoT by Region

Region	LTE Bands
EU	3, 8, 20
Commonwealth of Independent States	3, 8, 20
North America	2, 4, 5, 12, 66, 71, 26
Asia Pacific	1, 3, 5, 8, 18, 20, 26, 28
Sub-Saharan Africa	3, 8
Middle East and North Africa	8, 20
Latin America	2, 3, 5, 28

Frequency Range Detail (Europe)

NB-IoT Band	Uplink Band	Downlink Band	Bandwidth	Duplex Mode
-------------	-------------	---------------	-----------	-------------

3	1710 - 1785 MHz	1805 - 1880 MHz	75 MHz	HD-FDD
8	880 - 915 MHz	925 - 960 MHz	25 MHz	HD-FDD
20	832 - 862 MHz	791 - 821 MHz	30 MHz	HD-FDD

22.5.2 2.2. Deployment and Operation Modes

NB-IoT supports three primary operation modes for flexible deployment:

1. **Standalone:** This mode uses dedicated spectrum, often by “refarming” spectrum previously used by GSM networks. The NB-IoT signal occupies a 180 kHz channel within a 200 kHz GSM carrier, with 10 kHz guard bands on each side. It is ideal for areas without existing cellular network coverage and does not compete with mobile broadband services.
2. **In-band:** The NB-IoT signal occupies one 180-kHz Physical Resource Block (PRB) within a wider LTE carrier (typically 1.4–20 MHz). This is considered the most privileged mode due to its cost-effectiveness and ease of integration into legacy LTE networks. The specific PRBs are predefined to avoid interference with essential LTE channels.
3. **Guard-band:** The NB-IoT signal occupies an unused PRB within the guard band of an LTE carrier. This allows NB-IoT to coexist with primary LTE services without consuming LTE resources, though careful configuration is required to mitigate potential out-of-band interference. Guard-bands should be as small as possible.

22.5.3 2.3. Physical Layer (PHY)

The NB-IoT physical layer adheres to LTE design principles while being optimized for IoT use cases.

- **Multiple Access Scheme:** Orthogonal Frequency Division Multiple Access (OFDMA).
- **Modulation:** QPSK or BPSK.
- **Transmit Power:** 23 dBm.
- **Duplexing:** Frequency Division Duplex (FDD) mode only, with half-duplex operation (devices can only transmit or receive at any given time, not both).
- **Maximum Transport Block Size (TBS):** 1000 bits for uplink and 680 bits for downlink.

Frame Structure

- **Time Domain:** A 10 ms frame is divided into ten 1 ms subframes. Each subframe contains two 0.5 ms slots.
- **Frequency Domain:** The fundamental unit is a Physical Resource Block (PRB), which consists of 12 subcarriers with 15 kHz spacing, for a total bandwidth of 180 kHz.

The more symbol we assign, the better. More opportunities to transmit data. - Each symbol is separated by Guard- band to avoid inter symbol interference. ##### Uplink Transmission

	()	()	()	()	()	()
	*	*	*	*	*	*
	0.160183080767	0.160183080767	0.160183080767	0.160183080767	0.160183080767	0.160183080767
	KHz	(normal				
		mal				
		CP)				
	()	()	()	()	()	()
	*	*	*	*	*	*
	0.160183080767	0.160183080767	0.160183080767	0.160183080767	0.160183080767	0.160183080767
	KHz	(normal				
		mal				
		CP)				
	()	()	()	()	()	()
	*	*	*	*	*	*
	0.160183080767	0.160183080767	0.160183080767	0.160183080767	0.160183080767	0.160183080767
	KHz	(normal				
		mal				
		CP)				

22.5.4 2.4. MAC Layer Features

The MAC layer introduces features critical for extending battery life and ensuring reliable coverage.

Power-Saving Mode (PSM)

PSM allows a device to enter a deep-sleep state, disabling parts of its protocol stack to reduce power consumption to the micro-Ampere range. The device remains registered with the network, so a reattachment procedure is not required upon waking, saving significant energy. Two timers govern this mode:

- **PSM Active Timer (T3324):** The time a device stays in Idle Mode and is reachable for paging.
- **TAU Timer (T3412):** The time between Tracking Area Updates, defining the long sleep period.

Extended Idle Mode DRX (eDRX)

eDRX allows a device to enter a sleep mode while periodically waking to monitor the network for paging messages. The “eDRX cycle” can range from seconds to minutes. This is suitable for applications that need to receive data more frequently than PSM allows but do not require constant connectivity. eDRX can be used in conjunction with PSM, where the device performs eDRX cycles during the PSM Active Timer (T3324) period.

Sigfox	~ 162 dB
LTE-M	~ 160 dB
LoRa	~ 157 dB
LTE	~ 144 dB

22.6.2 3.2. Data Rate Capabilities

NB-IoT data rates are calculated based on the transport block size, the number of Resource Units (or subframes), the duration of these units, and scheduling overhead.

- **Uplink (Single-Tone):**
 - Minimum time to transmit 1000 bits: 48 ms.
 - Peak PHY Data Rate: 1000 bits / 48 ms = **20.8 Kbps**.
 - Effective Data Rate (with scheduling overhead): 1000 bits / (48+12) ms = **16.7 Kbps**.
- **Uplink (Multi-Tone):**
 - Minimum time to transmit 1000 bits: 4 ms.
 - Peak PHY Data Rate: 1000 bits / 4 ms = **250 Kbps**.
 - Effective Data Rate (with scheduling overhead): 1000 bits / (4+12) ms = **62.5 Kbps**.
- **Downlink:**
 - Minimum time to transmit 680 bits: 4 ms.
 - Peak PHY Data Rate: 680 bits / 4 ms = **170 Kbps**.
 - Effective Data Rate (with scheduling overhead): 680 bits / (4+22) ms = **26.2 Kbps**.

22.7 Executive Summary

The complex requirements of modern Internet of Things (IoT) applications—including real-time data exchange, large-scale deployments, and device constraints—necessitate specialized application-layer protocols that extend beyond the capabilities of standard TCP and UDP. While the HyperText Transfer Protocol (HTTP) forms the backbone of the traditional web, its strict, half-duplex, request-response model is ill-suited for the dynamic, event-driven nature of IoT. Key limitations of HTTP include its inability to support unsolicited server-side updates, its high overhead from verbose textual headers, and the complexity of options not required by many IoT use cases.

WebSocket emerges as a more efficient alternative, specifically designed to address these shortcomings. It establishes a persistent, full-duplex, bidirectional TCP connection between a client and a server through an initial HTTP handshake. This allows for low-latency, unsolicited data exchange from either endpoint with minimal overhead after the connection is established. The protocol's efficiency is further enhanced by a minimal frame header structure. However, despite its advantages for real-time communication, WebSocket remains fundamentally a client-server protocol. For certain large-scale IoT scenarios, more tailored solutions based on the publish-subscribe paradigm may prove more effective.

22.8 The Middleware Imperative in IoT

Standard internet transport protocols like TCP and UDP are foundational but insufficient on their own to manage the complexity of advanced IoT and Smart City applications. The unique demands of these environments require a “middleware” layer of application protocols and components to provide full support.

The need for this middleware is driven by several factors inherent to IoT systems:

- **Data Heterogeneity:** IoT networks must handle diverse types of data, including notifications, measurements, and commands.
- **Traffic Diversity:** The system must support both non-real-time traffic and traffic with hard real-time requirements.
- **Architectural Flexibility:** Both client-server and publish-subscribe communication models are often required within the same system.
- **Endpoint Constraints:** Many IoT devices have significant constraints related to processing power, battery life, and available bandwidth.
- **System Scale:** IoT deployments can be large-scale cyber-physical systems requiring robust and efficient management.
- **Storage Complexity:** The volume and velocity of data generated create significant storage challenges.

Given these varied and often conflicting requirements, no single networking solution can satisfy all IoT scenarios.

22.9 An Examination of HyperText Transfer Protocol (HTTP)

HTTP is the protocol that underpins the World Wide Web, defining how client-server programs retrieve web pages. A client (e.g., a browser) sends a request, and a server returns a response. It typically uses TCP as its transport protocol, leveraging TCP’s reliability to ensure messages are not lost or corrupted. When combined with Transport Layer Security (TLS), it becomes HTTPS (HTTP Secure).

22.9.1 Resource Identification: The URL

Resources on the web are identified by a Uniform Resource Locator (URL), which combines four key identifiers:

- **Protocol:** The client-server application to be used (e.g., `http`).
- **Host:** The unique name or IP address of the server.
- **Port:** The 16-bit port number for the application (e.g., HTTP defaults to port 80). It is often omitted if the default is used.
- **Path:** The location and name of the file on the server’s operating system (e.g., `/en/department/map-department`).

The standard formats are `protocol://host/path` or `protocol://host:port/path`.

() () () ()

* * * *

0.25.0501HEASIXes

for a response identical to a GET request, but without the response body.

() () () ()

* * * *

~~0.2501 POSTING No~~

mits an entity to the specified resource, often causing a change in state or side effects on the server.


```

() () ()
* * *

```

```

0.3333333333333333

```

```

(FIN) set
to
1,
in-
di-
ca-
tes
that
this
is
a
com-
ple-
te
mes-
sa-
ge
or
the
fi-
nal
frag-
ment
of
a
frag-
men-
ted
mes-
sa-
ge.

```

```

() () ()
* * *

```

```

0.3333333333333333

```

```

(Reserved)
sets reserved
for fu-
tu-
re
use.

```

() () ()
* * *

0.3333333333333333

bi-di-
ts ca-
tes
the
ty-
pe
of
mes-
sa-
ge
being
sent
(e.g.,
text,
bi-
na-
ry,
con-
nec-
tion
clo-
se).

() () ()
* * *

0.3.3.3.3 In-
(Length)

ts ca-
tes
the
pay-
load
length.
Its
in-
ter-
pre-
ta-
tion
is
va-
ria-
ble:

() () ()
* * *

0.3.3.3.3 Mask (Op-
tional)

A
ma-
sking
key
used
to
XOR
the
pay-
load
da-
ta.
Pre-
sent
on-
ly
if
M=1.

```

() () ()
* * *
0.3.3.3 Payload
Data ap-
table li-
ca-
tion
da-
ta.

```

22.10.3 Limitations in an IoT Context

While WebSocket significantly improves upon HTTP for real-time IoT communications, it has a primary limitation: it is still a client-server model. For many large-scale IoT systems, more effective and tailored solutions based on the publish-subscribe paradigm may be required.

22.11 Comparative Analysis: HTTP vs. WebSocket

The fundamental differences between the two protocols highlight WebSocket's suitability for modern, interactive applications.

```

() () ()
* * *
0.3.3.3 HTTP
tuSoc-
reket
() () ()
* * *
0.3.3.3 Connection
Type duplex,
per request-
si-response
stream-
code.
nec-
tion.
() () ()
* * *
0.3.3.3 Communication
di-di-
rec-
tidio-
nal

```

() () ()
* * *

0.3333333333333333
nigh
ma
afre-
terpea-
the
inire-
tiaquests.
hand-
sha-
ke.
() () ()
* * *

0.3333333333333333
ghde-
ra-
te

22.12 Executive Summary

Message Queuing Telemetry Transport (MQTT) is a lightweight, IoT-specific application protocol engineered for collecting event data from sensors in environments with constrained devices and potentially unreliable networks. Layered on TCP or TLS, it operates on a publish-subscribe paradigm, which decouples message producers (publishers) from consumers (subscribers) through a central broker. This architecture avoids direct client-to-client connections and routes messages based on subscriptions to specific “topics.”

Key features that define MQTT's utility in IoT include a highly efficient message frame with a minimal header, a flexible **Quality of Service (QoS)** mechanism offering three levels of message reliability, and specialized functionalities for state management. **Retained Messages** allow new subscribers to receive the most recent value for a topic immediately upon connecting. The **Last Will and Testament (LWT)** feature ensures system-wide notification if a client disconnects ungracefully. While MQTT's design makes it an excellent choice for embedded platforms, its reliance on a centralized broker can introduce challenges related to scalability and reliability.

22.13 1. Core Concepts and Architecture

MQTT is an application-layer protocol specifically designed for the Internet of Things, focusing on the efficient collection of data from sensors. Its architecture is fundamentally different from request-response protocols like HTTP.

22.13.1 Publish-Subscribe Paradigm

The MQTT protocol is implemented based on the publish-subscribe model, which involves three main components:

- **Publisher:** A client that sends messages.
- **Broker:** A central server that receives all messages from publishers and distributes them to the appropriate subscribers. It acts as the intermediary for all communication.
- **Subscriber:** A client that registers interest in one or more topics and receives messages published to those topics from the broker.

In this model, subscribers and publishers are decoupled; they do not need to know about each other's existence and never establish direct connections. Communication is organized around **topics**, which are hierarchical strings that act as message channels.

22.14 2. Message Frame Structure

To ensure efficiency on constrained devices, MQTT messages are characterized by a minimal header structure. The frame consists of a fixed header, an optional variable header, and an optional payload.

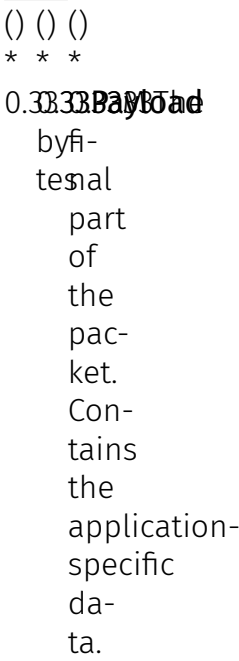
[illegible]

$$\begin{array}{ccc} \left(\right) & \left(\right) & \left(\right) \\ * & * & * \end{array}$$

0.3333333333333333 The
main length
of the
Length the
rest
of
the
pac-
ket
(Va-
ria-
ble
Hea-
der
+
Pay-
load).

$$\begin{array}{ccc} \left(\right) & \left(\right) & \left(\right) \\ * & * & * \end{array}$$

0.333 Variable
Header
field
present
in
some
mes-
sages.
Its
con-
tent
va-
ries
by
pac-
ket
ty-
pe.



22.14.1 Variable Byte Integer Encoding

The **Remaining Length** field uses a variable-length encoding scheme called a **Variable Byte Integer**. This allows the field to use between one and four bytes, depending on the size of the value it represents. In each byte, the lower 7 bits encode data, and the most significant bit (MSB) acts as a “continuation flag.” If the MSB is 1, it indicates that another byte follows; if it is 0, it is the final byte.

This encoding supports a maximum payload size of nearly 268 MB.

Bytes Used	Value Range From	Value Range To
1	0	127
2	128	16,383
3	16,384	2,097,151
4	2,097,152	268,435,455

22.15 3. Key MQTT Message Types

The protocol defines a set of messages for connection management, subscriptions, and data publication. The core messages examined are CONNECT, CONNACK, SUBSCRIBE, and PUBLISH.

22.15.1 CONNECT

The **CONNECT** message is the first packet sent by a client to open or reopen a session with the broker. It carries the client’s unique ID and various connection flags.

- **Key Fields in Variable Header:**

- **Protocol Name & Version:** Specifies the MQTT protocol details.
- **Flags:** A single byte containing several 1-bit flags:
 - * **Username & Password:** Indicate if credentials are present in the payload.
 - * **Will Flag:** If set, the “Will Message” fields (Will Topic, Will Payload, Will QoS, Will Retain) are included to configure the Last Will and Testament feature.
 - * **Clean Start:** If set, the broker creates a fresh session for the client. If not, it attempts to resume a previous session.
- **Keep Alive:** A 2-byte integer specifying the maximum time interval (in seconds) that can pass between control packets sent by the client. It is used to detect unresponsive clients.

22.15.2 CONNACK

The **CONNACK** message is the server's response to a **CONNECT** request, indicating the result of the connection attempt. It has no payload.

- **Key Fields in Variable Header:**
 - **Connect Flags:** Contains the `Session Present` flag, which informs the client whether the broker is resuming a pre-existing session.
 - **Reason Code:** A 1-byte code indicating the status of the connection.

() () ()
* * *

0.033333333333333333
**Read-
er descrip-
tion**

() () ()
* * *

0.033333333333333333
**The con-
nec-
tion
is ac-
cep-
ted.**

() () ()
* * *

0.3333333333333333 The
former
or
Pac
could
ket
not
par-
se
the
CON-
NECT
pac-
ket.

$$\begin{array}{ccc} () & () & () \\ * & * & * \end{array}$$

0.3333333333333333 The topac-coket Erwas ropar-sed, but its con-tent vio-la-tes the pro-to-col.

The super-powered Proot supports Verbs! The client's requested MQTT version.

0.3333333333333333
Ident
ti-ID
file
is
not syn-
vatic-
lidi-
cal-
ly
va-
lid
but
re-
jec-
ted
by
the
ser-
ver.

The client is banned from connecting (e.g., added to a blacklist).

22.15.3 SUBSCRIBE

The **SUBSCRIBE** message allows a client to register its interest in one or more topics. The broker will then forward messages published to these topics to the client.

- Key Fields in Payload:

- **Topic Filter(s):** A list of topic strings the client wishes to subscribe to.
- **Options:** Subscription-specific options for each topic filter.
 - * **QoS:** Specifies the maximum QoS level at which the server can send messages to this client for this subscription.
 - * **Retain Handling:** Dictates whether the server should send Retained Messages when the subscription is established.
 - * **Retain As Published:** Indicates if the Retain flag should be preserved on messages forwarded to this subscription.
 - * **No Local:** If set, the server will not forward messages to this client if this same client was the original publisher.

22.15.4 PUBLISH

The **PUBLISH** message is used to distribute application data. It can be sent from a client to the server (publishing a message) or from the server to a client (delivering a message).

- Key Fields:

- Control Flags:
 - * DUP: Set to 1 if the packet is a retransmission of an earlier message.
 - * QoS: Specifies the Quality of Service level for this message.

- * **Retain:** If set to 1, the broker must store this message as the “last known good” value for its topic.
- **Variable Header:**
 - * **Topic Name:** The topic channel to which the message is being published.
 - * **Packet Identifier:** A unique ID required for messages with QoS levels 1 and 2.
- **Payload:** The actual application message content.

22.16 4. Advanced Features and Mechanisms

MQTT includes several features designed to enhance reliability and state management in dynamic IoT environments.

22.16.1 Retained Messages

When a broker receives a **PUBLISH** message with the **Retain** flag set to 1, it stores that message for the specified topic. When a new client subscribes to that topic, the broker immediately sends this stored message to the new subscriber. This allows clients to get the latest status or value right away, without having to wait for the publisher to send a new update. This is particularly useful for sensors monitoring values that change infrequently.

22.16.2 Last Will and Testament (LWT)

The LWT is a mechanism for notifying other clients about an ungraceful disconnection. A client can specify a “Will Message,” “Will Topic,” and other parameters in its initial **CONNECT** packet. If the client’s connection terminates unexpectedly (e.g., without a clean **DISCONNECT** message or due to a protocol error), the broker will publish the Will Message to the Will Topic on behalf of the departed client. This feature can be combined with Retained Messages by setting the **Will Retain** flag, ensuring the LWT message is stored and delivered to future subscribers.

22.16.3 Keep Alive

The Keep Alive feature helps both the client and the server detect half-open TCP connections, where one side believes the connection is active while the other side has closed it. The client specifies a Keep Alive interval (in seconds) in the **CONNECT** message. The client is then responsible for ensuring it sends at least one MQTT packet within each interval. If the server does not receive any packets from the client within 1.5 times the Keep Alive interval, it assumes the connection is lost, closes it, and publishes the client’s LWT message if one was configured.

22.17 5. Quality of Service (QoS)

MQTT provides three distinct QoS levels to allow developers to choose the appropriate level of message delivery reliability for their application, which is crucial in unstable network environments. The QoS level is specified in the subscription request.

- **QoS 0: At Most Once**
 - Also known as “fire and forget.”
 - The message is sent once without any acknowledgment.
 - There is no guarantee of delivery, and messages may be lost.
- **QoS 1: At Least Once**
 - Guarantees that the message will be delivered at least one time.
 - The sender sends a **PUBLISH** packet and stores it until it receives a **PUBACK** (Publish Acknowledgment) packet from the receiver.
 - If no **PUBACK** is received, the message is retransmitted, which can lead to the receiver getting duplicate messages.
- **QoS 2: Exactly Once**
 - The highest level of service, ensuring that each message is received exactly once, without loss or duplication.
 - It involves a four-part handshake:
 1. Sender sends **PUBLISH** and waits for a **PUBREC** (Publish Received).
 2. Upon receiving **PUBREC**, the sender can discard the original message and sends a **PUBREL** (Publish Release).
 3. The receiver processes the message and responds with a **PUBCOMP** (Publish Complete).
 4. When the sender receives **PUBCOMP**, the transaction is complete.

22.18 6. Final Considerations

MQTT is a highly effective protocol for IoT systems, particularly those involving constrained endpoints like Arduino or Raspberry Pi.

22.18.1 Strengths

- **Lightweight:** The minimal message header makes it efficient for low-bandwidth networks and low-power devices.
- **Platform Support:** It can be implemented on many embedded platforms and has numerous open-source implementations available.
- **State Management:** The Retained Message mechanism allows for the seamless addition of new endpoints that can immediately synchronize with the system’s state.
- **Reliability Features:** The LWT feature provides robust management of connectivity outages.

22.18.2 Weaknesses

- **Centralized Broker:** The primary drawback is its reliance on a central broker. This can create a single point of failure and may pose scalability and reliability challenges in very large-scale deployments.

22.19 Executive Summary

The Constrained Application Protocol (CoAP) is an application layer protocol specifically engineered for the Internet of Things (IoT). It is designed to enable small, low-power devices with limited computational resources and bandwidth to participate in the broader internet. Operating on a client-server paradigm similar to HTTP, CoAP utilizes UDP for transport, necessitating its own mechanisms for error recovery. Security is managed at the network layer via IPsec or through Datagram Transport Layer Security (DTLS).

Key features of CoAP include its compact message format, interoperability with HTTP via gateways, and a unique “resource observation” function that emulates a publish-subscribe model, allowing clients to receive asynchronous updates on a resource’s state. The protocol distinguishes between **Message IDs**, used for duplicate detection and acknowledging messages, and **Tokens**, used to match specific requests with their corresponding responses. While its RESTful model and simplicity are advantageous for web developers, its fundamental client-server nature makes MQTT a more suitable choice for applications requiring a true publish-subscribe architecture.

22.20 1. Core Concepts and Architecture

22.20.1 1.1. Protocol Purpose and Design

CoAP is an IoT-specific application protocol created to accommodate the limitations of constrained devices, such as scarce bandwidth, low power, limited computational capacity, and long sleep times. Its primary goal is to allow these devices to integrate with the wider internet.

- **Transport Protocol:** CoAP is layered on top of the User Datagram Protocol (UDP), in contrast to protocols like MQTT which typically use TCP. This choice necessitates dedicated error recovery mechanisms within CoAP itself.
- **Security:** To provide security, CoAP can utilize IPsec at the network layer or Datagram Transport Layer Security (DTLS).
- **Communication Model:** It is implemented based on the client-server paradigm, with communication consisting of a sequence of requests (methods) and responses, conceptually similar to HTTP. Although not equivalent to HTTP, inter-networking between the two protocols is designed to be straightforward.

22.20.2 1.2. System Architecture

CoAP supports several communication patterns:

- Endpoints (clients) can communicate with a CoAP server.
- Endpoints can communicate with an HTTP server through a gateway.
- Endpoints can communicate directly in a peer-to-peer fashion.

A significant feature is that the “lightweight” characteristics of the protocol make it feasible to configure a CoAP server even on resource-constrained endpoints. Furthermore, a device acting as a CoAP server may also act as a client.

22.21 2. CoAP Frame Structure and Semantics

The CoAP message format is designed to be more compact than HTTP’s, though it can be slightly more complex than MQTT’s.

Header Format:

V (2 bits) | T (2 bits) | TKL (4 bits) | Code (1 byte) | Message ID (2 bytes)
Token (0-8 bytes) Options (N bytes) Payload (K bytes)

22.21.1 2.1. Header Fields Explained

- **Version (V):** 2 bits indicating the CoAP version.
- **Type (T):** 2 bits defining the message type.
 - **0 - Confirmable (CON):** This message requires an acknowledgement message.
 - **1 - Non-confirmable (NON):** This message does not require confirmation.
 - **2 - Acknowledgement (ACK):** A response that acknowledges a Confirmable message.
 - **3 - Reset (RST):** Indicates the receiver cannot or will not process a request.
- **Token Length (TKL):** 4 bits specifying the length of the Token field, which can be 0.
- **Code:** 1 byte indicating the request method or response code. It consists of a 3-bit class and a 5-bit code.
- **Message ID:** A 2-byte value used to detect duplicate messages and retransmissions.
- **Token:** A variable-length field (0-8 bytes) used to correlate a request with its response.

22.21.2 2.2. Distinguishing Message ID vs. Token

CoAP uses two separate identifiers for message correlation, each with a distinct purpose.

- **Message ID:** This unique value is used to match a **Confirmable (CON)** message to its corresponding **Acknowledgement (ACK)** message. Its primary role is in ensuring message delivery and detecting duplicates.

- **Token:** This unique value is used to match a **Request** to its **Response**. This is necessary because a response may be sent in a different message than the ACK.

Response Delivery Mechanisms

1. **Piggybacked Response:** The response is sent directly within the ACK message for a Confirmable request. In this case, the message has the same Message ID and Token as the original request. This is efficient as it does not require a separate ACK from the client.
2. **Separate Response:** The server first sends an empty ACK to confirm receipt of the request and later sends the full response in a separate message. This is useful if generating the response takes time. The separate response will have the same Token as the request but a different Message ID, and as a new Confirmable message, it will require its own ACK.

The Token may be omitted in Non-confirmable mode or if there is only a single pending request between endpoints where no ambiguity exists.

22.22 3. Communication: Requests and Responses

22.22.1 3.1. Request Methods

CoAP defines several request methods, identified by a code, that are analogous to those in HTTP.

```

_
() ()
* *
0.505000Ac-
thid/Request
(Co-
de)
() ()
* *
0.505000GET
(0.00)
the
sta-
te
of
a
re-
sour-
ce.
```

$$\overline{\quad}$$

$$\begin{pmatrix} & \end{pmatrix} \quad \begin{pmatrix} & \end{pmatrix}$$

$$\begin{matrix} * & * \end{matrix}$$

0.50500POST-

(0.02) tests

the processing of the message content, potentially creating or updating a resource.

```

() ()
* *
0.5050005 Sub-
(0.03)
tu-
tes
the
re-
sour-
ce
spe-
ci-
fied
in
the
URI
or
crea-
tes
a
new
one
if
it
does
not
exi-
st.
() ()
* *
0.5050005 CRE-
(0.04)
ves
the
re-
sour-
ce.

```

22.22.2 3.2. Response Codes

Responses are characterized by codes that indicate the outcome of a request.

```
( ) ( ) ( )
* * *
0.3333333333333333Ac-
tion
se
Class
( ) ( ) ( )
* * *
0.3333333333333333SUCCESS
(2.xx)
( ) ( ) ( )
* * *
0.3333333333333333Created:
The
re-
sour-
ce
has
been
suc-
ces-
sul-
ly
crea-
ted.
( ) ( ) ( )
* * *
0.3333333333333333Deleted:
The
re-
sour-
ce
has
been
suc-
ces-
sul-
ly
de-
le-
ted.
```

() () ()
* * *

0.333333 Content:

The
mes-
sa-
ge
car-
ries
the
con-
tent
(pay-
load)
of
the
re-
que-
sted
re-
sour-
ce.

() () ()
* * *

0.333333 CLIENT

ER-
ROR
(4.xx)

() ()
* * *

0.3333333333333333
**SERVER-
ERROR**
(5.xx)
() ()
* * *

0.3333333333333333
Not Im-
p-
ple-
men-
ted:
The
ser-
ver
can-
not
ful-
fill
the
re-
que-
st
(e.g.,
does
not
re-
co-
gni-
ze
the
me-
thod).

22.23 4. Key Functionality: Resource Observation

A powerful feature of CoAP is **resource observation**, which allows a device to be asynchronously updated on the state of a resource, similar to a publish-subscribe model. This is particularly useful in IoT for unsolicited sensory measurements.

- **Initiating Observation:** A client sends a GET message with the “Observer” option set to 0.
- **Receiving Updates:** The “observer” client will then receive a sequence of responses from the server. All these update messages will carry the same Token as the original request. Each response includes a sequence number that increa-

ses with each update, allowing the client to reorder messages if they arrive out of order.

- **Update Rate:** The rate of updates depends on the nature of the resource. For example, a temperature sensor resource defined with a 1°C resolution will generate fewer updates than one defined with a 0.1°C resolution for the same physical sensor. Rate-limiting mechanisms can be implemented to prevent message flooding.
- **Terminating Observation:** The observation ends when the server sends a `Reset` message or when the client sends a new `GET` message for the same resource with the “Observer” option set to 1.

22.24 5. Final Considerations and Comparison with MQTT

22.24.1 5.1. Advantages and Drawbacks

CoAP offers several advantages that make it a strong competitor to MQTT:

- **Simple Design:** Well-suited for IoT networks with many resource-constrained nodes.
- **Asynchronous Support:** The “Observer” method supports asynchronous events where strict real-time guarantees are not required.
- **HTTP Alignment:** Interoperability with HTTP and its REST model makes it easier for web designers to build CoAP applications.

However, it also has drawbacks:

- **Client-Server Model:** Despite the “Observer” feature, CoAP remains a client-server protocol. For applications that require a true, robust publish-subscribe paradigm, MQTT is generally more convenient.

22.24.2 5.2. CoAP vs. MQTT Comparison

The following table provides a direct comparison of key features between MQTT and CoAP.

	MQTT	CoAP
Version	3.1.1	0.3.3
Protocol	TCP	UDP
Message Type	Single	Single/Broadcast
QoS	1/2	0/1
Security	SSL/TLS	DTLS
Authentication	Username/Password	Pre-shared Key
Retain	Yes	No
Wildcard	Yes	No
Subscription	Yes	No
Observation	No	Yes
Reset	No	Yes
Message Size	2 MB	256 KB
Message ID	Yes	No
Wildcard	Yes	No
Subscription	Yes	No
Observation	No	Yes
Reset	No	Yes
Message Size	2 MB	256 KB
Message ID	Yes	No

() () ()
* * *

0.3.3.3 Purpose

sasi-
gined
and
comsource-
mconstrained
ni-de-
cavi-
tions
inin
loToT

() () ()
* * *

0.3.3.3 Transport

Pr(Fuser
toDa-
cosida-

Cogr
troPro-
Pro-
to-
col)
col)

() () ()
* * *

0.3.3.3 Communication

StyHighSubscribe
lemt/Response

de(Client-
Server-
like)
mo-
del

() () ()
* * *

0.3.3.3 Reader

Si-byby-
zetes
fi-fi-
xed
hea-
der

() () ()

* * *

0.3333333333333333

For ports

mātibī-

naaa-

ry ry

and

text

pay-

loads

() () ()

* * *

0.3333333333333333 Re-

velsa-

0, bi-

1. li-

antiv

2 th-

forou-

most

sa“con-

gefir-

dem a-

li-ble”

yeand

ry "non-

confirmable"

mes-

sa-

ges

() () ()

() () ()
* * *

~~0.30.30.30 Message~~

Työliikenne-

pe\$u\$F,

SCRIPT.

beDE-

ColnE-

netE.

et cetera.

() () ()
* * *

0.333 Resource-
Din-in
score-
verestur-
ryre-ce
qudi-
resco-
exve-
tray
meh-
chou-
nigh
smCo-
RE
Link
For-
mat

22.25 1.0 Introduction: The Industrial Imperative for a Modified Ethernet

While standard Ethernet has become the dominant wired communication technology in enterprise and consumer environments, its native architecture is fundamentally unsuited for the stringent demands of industrial automation and control. Its widespread success is rooted in its cost-effectiveness, broad market adoption, and the easy availability of skilled personnel. However, the protocol's design priorities—flexibility, ease of use, and cost—are often at odds with the industrial requirements for precision, predictability, and operational safety.

The motivation to adapt Ethernet for the factory floor is compelling. Leveraging this ubiquitous technology offers significant advantages that industrial operators cannot ignore:

- **Cost Reductions:** The enormous production scale of Ethernet components leads to significantly lower hardware costs compared to specialized industrial fieldbuses.
- **Skilled Personnel:** A vast pool of IT professionals is already trained in Ethernet networking, simplifying installation, maintenance, and troubleshooting.
- **Network Homogeneity:** The potential exists to create a unique and homogeneous network technology that seamlessly connects the factory floor (Operational Technology, OT) with enterprise-level IT systems.

The central problem with standard Ethernet is its lack of determinism. The protocol was not designed to guarantee that a data frame will arrive at its destination within a precise, predictable timeframe. This non-deterministic nature is unacceptable for

controlling machinery, where a delay of even a few milliseconds can lead to production errors, equipment damage, or safety failures. To overcome these limitations, a family of solutions known as “Industrial Ethernet” or “Real-Time Ethernet (RTH)” has been developed, modifying the standard protocol to meet the real-time demands of the industrial world.

To fully appreciate the innovations of Industrial Ethernet, it is first necessary to review the foundational principles and inherent limitations of the original IEEE 802.3 standard that necessitated these changes.

22.26 2.0 Foundational Principles of Standard Ethernet (IEEE 802.3)

This section establishes a technical baseline by examining the core services, frame structure, and media access control mechanism of standard Ethernet (IEEE 802.3). By analyzing these foundational elements, we can identify the specific characteristics that pose significant challenges in deterministic industrial environments and understand the rationale behind the modifications introduced by Industrial Ethernet protocols.

22.26.1 2.1 Fundamental Protocol Services

The Ethernet data-link layer provides a set of services that, while effective for general data communication, introduce unpredictability in time-critical applications.

- **Connectionless Service:** Frames are transmitted whenever the application layer has data ready, without a pre-established connection. This simplicity can lead to network congestion if multiple devices attempt to transmit simultaneously.
- **No Flow Control:** The protocol provides no native mechanism to prevent a fast sender from overwhelming a slower receiver. When a receiver’s buffer is full, excess frames are simply discarded without notification, leaving loss recovery to higher-level protocols like TCP, which adds its own latency.
- **No Acknowledgement (ACK):** While frames include a Cyclic Redundancy Check (CRC) field for error detection, the MAC layer does not implement an Automatic Repeat Request (ARQ) mechanism. If a receiver detects a corrupted frame (via a failed CRC), it is silently dropped.
- **Collision-Based Retransmission:** In its original half-duplex form, Ethernet uses Carrier-Sense Multiple Access with Collision Detection (CSMA/CD) to manage access to a shared transmission medium. This mechanism is a primary source of non-determinism, as collisions force devices to back off and attempt retransmission after a randomized delay.

22.26.2 2.2 Rationale for Frame Size Limitations

The constraints on Ethernet’s frame size are a direct result of its historical context and the physics of its media access control mechanism.

The **maximum frame length** was set to 1518 bytes (including a 1500-byte payload) for several strategic reasons dating back to the 1980s. This limit prevents a single station from monopolizing the shared network bus for an extended period, bounds the probability of a bit error occurring within a single frame, and crucially, reduced the amount of expensive buffer memory required in early network interface cards.

Conversely, the **minimum frame length** of 64 bytes is a direct consequence of the CSMA/CD mechanism. For collision detection to function reliably, the time it takes to transmit the smallest possible frame (T_{fr}) must be greater than twice the maximum signal propagation delay across the network (T_p). This ensures that a transmitting station is still listening for a collision signal when one might arrive from the furthest end of the network. This relationship can be expressed as $T_{fr} > 2 * T_p$.

Using the parameters from the original Ethernet standard:

- Signal propagation speed over copper cable (c) is approximately $2 * 10^8$ m/s.
- Maximum network diameter (d) was designed to be 2500 meters.
- Standard bitrate (C) was 10 Mbps.

The one-way propagation delay is calculated as $T_p = d / c = 2500 \text{ m} / (2 * 10^8 \text{ m/s}) = 12.5 \text{ } \mu\text{s}$. Therefore, the round-trip delay is $2 * T_p = 25 \text{ } \mu\text{s}$. The minimum frame size (S_{min}) must be large enough that its transmission time is at least this long: $S_{min} = C * (2 * T_p) = 10 \text{ Mbps} * 25 \text{ } \mu\text{s} = 250 \text{ bits}$, or approximately 31.25 bytes. To account for delays introduced by repeaters and to add a safety margin, the standard was set to 64 bytes.

These foundational characteristics—particularly the probabilistic nature of CSMA/CD and the lack of real-time guarantees—created the definitive need for a new class of Ethernet protocols designed specifically for the factory floor.

22.27 3.0 The Evolution to Industrial Ethernet: Achieving Determinism and Real-Time Performance

Industrial Ethernet is not a single, monolithic standard but rather a broad evolution of standard Ethernet technology. It encompasses a range of protocols and hardware modifications designed to provide the reliability, determinism, robustness, and real-time communication capabilities essential for controlling machines and production processes. These technologies are often categorized into classes based on the extent of their modifications and their resulting real-time performance.

() () ()
* * *

0.3333333333333333
niTime
caCa-
Appa-
proach
li-
ty
&
Li-
mi-
ta-
tions

() () ()
* * *

0.3333333333333333 Pro-

1 li-vi-
zes
the“soft”
sta-real-
d-time
TCP/IP
sta-fok-
over-man-
or-ce
di-with
na-la-
ry-ten-
Eth-ers
netty-
hard-
ware-
ly
≥
100
ms.
Pro-
ne
to
un-
pre-
dic-
ta-
ble
de-
lays
due
to
net-
work
con-
ge-
stion
and
stan-
dard
pro-
to-
col
ove-
rhead.

() () ()
* * *

0.3.3.3 Class A Achie-

2 places
mem-
ts pro-
a ved
cula-
stem-
real-
time-
pro-
total-
coly
stack
di-10
rems).
tlyCan-
onnot
theou-
sta-
dard-
Ether-
nets
layer,
byrou-
pagh
singan-
TCARD
forIP
time-
critical,
daas
ta.it
does
not
use
the
IP
layer
for
real-
time
traf-
fic.

() () ()
* * *

0.3333333333333333

3 plays
a vers
mohard"
di-real-
time
Ethper-
netfor-
proman-
toce
cowith
and-
reten-
quies
res
cu1
stons.
had-
ware
tothe
furni-
daghe-
met-
talle-
lyvel
al-of
tede-
how-
frani-
mes-
arem
prbut
cess
sech-
com-
pa-
ti-
ble
with
stan-
dard
IP
rou-
ting
and
equi-
p-
ment.

22.27.1 3.3 Dedicate a subsection to the critical enabling technology for synchronization

3.3.1. Clock Synchronization with IEEE 1588 Precision Time Protocol (PTP)

A primary barrier to early Industrial Ethernet adoption was the lack of a common timing reference across all network devices, which is critical for synchronized operations like motion control. Solutions like GPS are often unsuitable for industrial settings due to their inability to function indoors and potential inaccuracies. The solution was to use the network itself to distribute highly precise timing signals.

The **IEEE 1588 Precision Time Protocol (PTP)** is a network-based method that allows a master clock to synchronize all slave clocks on a network with sub-microsecond accuracy. The process corrects for both the time offset between clocks and the variable network transmission delay.

The PTP synchronization process can be illustrated with the following step-by-step example:

1. **Initial State:** The Master clock reads 100s while the Slave clock, with a -20s offset, reads 80s.
2. **Sync Message:** At time 101s, the Master sends a **Sync** message.
3. **Sync Arrival:** The **Sync** message travels for 2s and arrives at the Slave when the Slave's clock reads 83s. (The corresponding Master time would be 103s).
4. **Follow_Up Message:** The Master sends a **Follow_Up** message containing the exact timestamp of when the **Sync** message was sent (101s).
5. **Offset Calculation:** The Slave receives the **Follow_Up** message and compares the Master's send timestamp (101s) with its own arrival timestamp (83s).
6. **Initial Adjustment:** The Slave calculates the difference: $101s - 83s = 18s$. This value represents the perceived offset, which includes both the true clock offset and the network delay.
7. **Clock Correction (Part 1):** The Slave applies this correction to its current time. Its clock, which has advanced to 85s, is adjusted to $85s + 18s = 103s$. At this point, the clocks are running in parallel but are not yet fully synchronized; the slave's clock still lags the master's by the one-way network delay.
8. **Delay_Req Message:** To measure this delay, the Slave sends a **Delay_Req** message at its new time of 108s.
9. **Delay_Req Arrival:** The message arrives at the Master at Master time 112s.
10. **Delay_Resp Message:** The Master immediately sends a **Delay_Resp** message back to the Slave, containing the arrival timestamp (112s).
11. **Delay Calculation:** The Slave receives the **Delay_Resp** at its local time 112s. It now has enough information to calculate the round-trip time: the total time elapsed from sending its request (108s) to receiving the master's timestamped response (112s) is $112s - 108s = 4s$.
12. **Final Correction Logic:** The Slave deduces that the one-way network delay is half the round-trip time: $4s / 2 = 2s$. It now knows two critical facts: its clock is running parallel to the master's, and the transmission delay is 2 seconds.
13. **Full Synchronization:** When the **Delay_Resp** arrived at the Slave's local time of 112s, the message had been in transit for 2 seconds. Therefore, the Master's clock must have been 114s at that exact moment. The Slave performs a final

alignment, adjusting its clock from 112s to 114s, achieving full synchronization with the Master.

22.27.2 3.4 Create a subsection to analyze the physical layer adaptation

3.4.1. Physical Layer Adaptation: Single Pair Ethernet (SPE)

Beyond protocol-level modifications, the physical wiring of Ethernet also required adaptation for industrial scenarios. **Single Pair Ethernet (SPE)** addresses key industrial constraints, such as limited space in control cabinets and the need for lower power consumption for simple sensors and actuators.

—
() (
* *
0.505000Com-
tripa-
buri-
teson
to
Tra-
di-
tio-
nal
Ether-
net
() (
* *
0.505000Wiring
a
sin-
gle
twi-
sted
pair
(two
wi-
res)
in-
stead
of
four
pairs
(eight
wi-
res).

* *

0.505000Data p-

Reports

te ra-

tes

from

10

Mbps

to

1

Gbps,

com-

pa-

red

to

up

to

100

Gbps

for

tra-

di-

tio-

nal

Ether-

net.

0.5000 Range

tends reach up to 1 km, significantly longer than the standard 100 m limit.

0.500000Cost-

motive use cases:

0.25 0.5 0.75 1.0 Pri-

dardstama-
cery
Use
Ca-
se

~~() () () ()~~
~~* * * *~~

~~0.25000BASE-Long-~~

~~T1Mbps~~ reach

ap-
pli-
ca-
tions
in
build-
ing
and
pro-
cess
au-
to-
ma-
tion.

~~() () () ()~~
~~* * * *~~

~~0.25000BASE-Multi-~~

~~T1Mbps~~ drop

to-
po-
lo-
gies
for
au-
to-
mo-
ti-
ve
and
in-
machine
net-
wor-
king.

() () () ()
* * * *

0.25 Mbps
T1Mbps speed

au-
to-
mo-
ti-
ve
and
in-
du-
strial
de-
vi-
ce
con-
nec-
ti-
vi-
ty.

() () () ()
* * * *

0.25 Mbps
T1Gbps

ga-
bit
con-
nec-
ti-
vi-
ty
for
au-
to-
mo-
ti-
ve
ca-
me-
ras
and
sen-
sors.

() () ()
 * * * *
 0.25msec/100-
 T110m ry
 Gbps high-
 speed
 links
 for
 Li-
 DAR,
 ra-
 dar,
 and
 AI
 ca-
 me-
 ras.

While these classes and enabling technologies define the broader landscape of Industrial Ethernet, specific protocols like EtherCAT have implemented these concepts in unique and highly optimized ways to achieve best-in-class performance, which will be explored next.

22.28 4.0 Deep Dive: EtherCAT Architecture and Operational Mechanisms

EtherCAT (Ethernet for Control Automation Technology) is a high-performance, Class 3 Industrial Ethernet standard renowned for its exceptional speed and determinism. It is particularly suited for hard real-time applications such as robotics, multi-axis motion control, and high-performance drives. This performance is not achieved by simply layering a real-time protocol on top of Ethernet, but through fundamental modifications to Ethernet’s architecture and data processing model.

22.28.1 4.1 “On the Fly” Processing

The core architectural principle of EtherCAT is its “on the fly” processing model, which directly addresses the sources of non-determinism—namely, store-and-forward latency and switch jitter—found in standard switched Ethernet.

In a conventional network, switches introduce variable latency as they receive, store, process, and forward each frame. EtherCAT replaces this topology with a master-slave daisy-chain ring. The EtherCAT master is the only device that initiates frame transmission. The frame travels sequentially through each slave device, which processes its relevant data *as the frame passes through it*. Each slave reads the data designated for it and inserts its response data into the same frame, all with minimal hardware-level delay. Because slaves do not need to buffer and forward the entire frame, this

mechanism virtually eliminates switch jitter and ensures highly deterministic communication cycles, often as low as 100 μ s or less. ### 4.2 Deconstruct the EtherCAT

Missing Image: Drawing 2025-12-03 08.59.51.excalidraw

Frame Structure

EtherCAT telegrams are not a new packet type but are encapsulated directly within a standard Ethernet II frame, using a dedicated EtherType value. This allows EtherCAT traffic to be transported over standard Ethernet physical layers. The structure is hierarchical:

- **Ethernet Frame:** The standard outer container with destination/source MAC addresses.
- **EtherCAT Frame:** A simple 2-byte header containing the total length of the encapsulated EtherCAT data.
- **EtherCAT Telegram:** A container for one or more EtherCAT datagrams that are processed sequentially.
- **EtherCAT Datagrams:** The individual command and data payloads for the slaves. Each datagram consists of a 10-byte header, a data payload, and a 2-byte Working Counter trailer.

The **EtherCAT datagram header** is critical to the protocol's function. Key fields include:

The **Working Counter** is a 2-byte trailer that follows the data payload of each datagram. Each slave that successfully processes the datagram increments this counter. When the frame returns to the master, the master can verify the final value of the Working Counter against the expected value to confirm that all addressed slaves successfully executed their commands.

22.28.2 4.3 Evaluate EtherCAT's Addressing and Memory Mapping Mechanisms

The "on the fly" processing is enabled by custom hardware in each slave device, primarily the **EtherCAT Slave Controller (ESC)** and its **Fieldbus Memory Management Unit (FMMU)**. The ESC receives the passing frame and, guided by the FMMU, extracts or inserts data directly from or into the frame without software intervention. EtherCAT uses four distinct addressing methods to manage communication:

1. **Position-based (Auto-Increment) Addressing:** Used primarily during network start-up. The master sends a broadcast command, and each slave in the chain decrements a counter as the frame passes. The slave whose counter reaches zero is addressed, allowing the master to discover the network topology and assign unique addresses.
2. **Configured Addressing:** After start-up, the master assigns a unique 16-bit fixed address to each slave. This method is used for non-cyclic, direct master-to-slave communication, such as for configuration or diagnostics.
3. **Logical Addressing:** This is the primary method for high-speed, cyclic communication. Instead of addressing individual slaves, it addresses data locations within the frame itself. The master constructs a single, large "process image"

containing all input and output data for the entire network segment. The FMMU in each slave is configured to map specific bytes from this logical process image to physical memory addresses within its ESC, allowing it to read its inputs and write its outputs efficiently.

4. **Broadcast Addressing:** Allows the master to send a command that is received and processed by all slaves simultaneously. This is used for tasks like network-wide initialization or error recovery.

EtherCAT's unique architecture provides exceptional performance for specific, demanding applications. These features will be placed in a broader comparative context in the final section.

22.29 5.0 Comparative Summary and Application Suitability

This analysis has deconstructed the evolution from standard, non-deterministic Ethernet to the specialized, real-time protocols required for industrial automation. The following summary synthesizes these technical details, offering high-level guidance on the suitability of each technological approach for distinct industrial scenarios.

() () () ()
* * * *

0.25ms
Key
no
eter
Ar-
lo-
Menichi-
gycha
tec-
nism
tu-
sm
& ral
Rela-
Time
Cares
pa-
bi-
li-
ty

() () () ()
 * * * *

0.25 Mbps/100 m
 Ethernet
 net
 (IEEE 802.3)
 Full-duplex
 Duplex
 Switch
 Store-and-forward
 frame processing; relies on higher-level protocols for reliability.

```

() () () ()
* * * *
0.25mmms60f1
Ethernet-
netEthe-
(Classiges
sesarestan-
1 wanard,
& withow-
2) speost
cianshard-
li-Deware;
zetuses
proipro-
tonito-
cosmols
layis-
(TSoreware-
orbased
cuan1588
stonATP
realefor
timeclock
stacksyn-
techro-
byni-
neta-
wdrn.
load.

```

```

() () () ()
* * * *
0.25ms/segment
(Class 1)
3) withing
“on-to-
thghy-
flydelo-
proty;
cenisum-
sing.ma-
stition
witha-
cymes
clepro-
ti-cess
mech-
≤100
μsfor
mul-
ti-
ple
sla-
ves
in
one
pass;
re-
qui-
res
cu-
stom
sla-
ve
hard-
ware
(ESC/FMMU).

```

In conclusion, the choice of Ethernet technology must align directly with the application’s performance requirements. For monitoring, data acquisition, and non-critical control tasks, **Class 1 and 2 Industrial Ethernet** protocols offer a compelling balance of performance, high compatibility with standard IT infrastructure, and ease of integration. They leverage the cost benefits of standard hardware while adding sufficient real-time capability through software and synchronization protocols. In stark contrast, for applications where precision, speed, and safety are paramount—such as multi-axis motion control, robotics, and high-speed packaging machines—the ultra-low latency and high-precision synchronization of a **Class 3 system like EtherCAT** are essential. Its

unique hardware-based “on the fly” processing model provides a level of determinism that software-based approaches cannot match, making it the definitive choice for the most demanding industrial control challenges.